

Aleph: Efficient Atomic Broadcast in Asynchronous Networks with Byzantine Nodes

Adam Gagol^{1,2}, Damian Leśniak^{1,2}, Damian Straszak¹, Michał Świętek^{1,2}

¹ Aleph Zero Foundation

² Jagiellonian University

ABSTRACT

The spectacular success of Bitcoin and Blockchain Technology in recent years has provided enough evidence that a widespread adoption of a common cryptocurrency system is not merely a distant vision, but a scenario that might come true in the near future. However, the presence of Bitcoin's obvious shortcomings such as excessive electricity consumption, unsatisfying transaction throughput, and large validation time (latency) makes it clear that a new, more efficient system is needed.

We propose a protocol in which a set of nodes maintains and updates a linear ordering of transactions that are being submitted by users. Virtually every cryptocurrency system has such a protocol at its core, and it is the efficiency of this protocol that determines the overall throughput and latency of the system. We develop our protocol on the grounds of the well-established field of Asynchronous Byzantine Fault Tolerant (ABFT) systems. This allows us to formally reason about correctness, efficiency, and security in the strictest possible model, and thus convincingly prove the overall robustness of our solution.

Our protocol improves upon the state-of-the-art HoneyBadgerBFT by Miller *et al.* by reducing the asymptotic latency while matching the optimal communication complexity. Furthermore, in contrast to the above, our protocol does not require a trusted dealer thanks to a novel implementation of a trustless ABFT Randomness Beacon.

KEYWORDS

Byzantine Fault Tolerance, Asynchrony, DAG, Atomic Broadcast, Randomness Beacon, Consensus, Cryptography

1 INTRODUCTION

The introduction of Bitcoin and the Blockchain in the seminal paper of Satoshi Nakamoto [39] is already considered a pivotal point in the history of Financial Technologies. While the rise of Bitcoin's popularity clearly shows that there is significant interest in a globally distributed currency system, the scalability issues have become a significant hurdle to achieve it. Indeed, Bitcoin's latency of 30 to 60 minutes, the throughput of 7 transactions per second, and the excessive power usage of the proof of work consensus protocol have motivated the search for alternatives.

At the core of virtually every cryptocurrency system lies a mechanism that collects transactions from users and constructs a total ordering of them, i.e., either explicitly or implicitly forming a blockchain of transactions. This total ordering is then used to determine which transaction came first in case of double-spending attempts and thus to decide which transactions should be validated. The protocol that guides the maintenance and growth of

this total ordering is the heart of the whole system. In Bitcoin, the protocol is Proof of Work, but there are also systems based on Proof of Stake [12, 30] and modifications of these two basic paradigms [32, 44]. Aside from efficiency, the primary concern when designing such protocols is their security. While Bitcoin's security certainly has passed the test of time, numerous newly proposed designs claim security but fall short of providing convincing arguments. In many such cases, serious vulnerabilities have been discovered, see [2, 18].

Given these examples, one may agree that for a new system to be trusted, strong mathematical foundations should guarantee its security. What becomes important then are the assumptions under which the security claim is pursued – in order to best imitate the highly adversarial execution environment of a typical permissionless blockchain system, one should work in the strictest possible model. Such a model – the Asynchronous Network model with Byzantine Faults – has spawned a large volume of research within the field of Distributed Systems for the past four decades. **Protocols that are designed to work in this model are called Asynchronous Byzantine Fault Tolerant (ABFT) – and are resistant to harsh network conditions: arbitrarily long delays on messages, node crashes, or even multiple nodes colluding in order to break the system.** Interestingly, even though these protocols seem to perfectly meet the robustness requirements for these kinds of applications, they have still not gained much recognition in the crypto-community. **This is perhaps because the ABFT model is often considered purely theoretical, and in fact, the literature might be hard to penetrate by an inexperienced reader due to heavy mathematical formalism.** Indeed, several of the most important results in this area [9, 10, 19, 24] have been developed in the '80s and '90s and were likely not meant for deployment at that time but rather to obtain best asymptotic guarantees. Now, 30 years in the future, perhaps surprisingly, the ABFT model has become more practically relevant than ever, since the presence of bad actors in modern distributed ledger systems is inevitable, and their power ranges from blocking or taking over several nodes to even entirely shutting down large parts of the network.

In recent important work [35], Miller *et al.* presented the HoneyBadgerBFT (HBBFT) protocol, taking the first step towards practical ABFT systems. HBBFT achieves optimal, constant communication overhead, and its validation time scales logarithmically with the number of nodes. Moreover, importantly, HBBFT is rather simple to understand, and its efficiency has been also confirmed by running large-scale experiments. Still, an unpleasant drawback of HBBFT, especially in the context of trustless applications, is that it requires a trusted dealer to initialize.

In this paper, we present a completely new ABFT protocol that keeps all of the good properties of HBBFT and improves upon it

in two important aspects: **tightening the complexity bounds on latency from logarithmic to constant and eliminating the need for a trusted dealer**. Furthermore even though being developed for the asynchronous setting, **it matches the optimistic-case performance of 3-round validation time of state-of-the-art synchronous protocols [21]**. We believe that our protocol is **simple to understand, due to its transparent structure that clearly separates the network layer from the protocol logic**. We also present a contribution that might be of independent interest: an efficient, completely trustless ABFT Randomness Beacon that generates common, unpredictable randomness. Since such a mechanism is necessary in many blockchain-based systems, we believe it might see future applications. Finally, we believe that this paper, while offering a valuable theoretical improvement, also contributes to bridging the gap between the theory and practice of ABFT systems.

2 OUR RESULTS

The main goal of this paper¹ is to design a distributed system that runs in a trustless network environment and whose purpose is to build a collective total ordering of messages submitted by users. Apart from blockchain, such systems have several other applications, for example implementing state machine replication, where messages can be arbitrary operations to be executed by a state machine, and the purpose of the system is to keep the states of several copies of the machine consistent by executing the commands in the same order.

A major decision to make when designing systems of this kind is how to realistically model a network environment where such a system would run. In the subsequent paragraphs, we introduce the model we work in and argue why we find it the most suitable for applications in distributed financial systems.

Nodes and Messages. The system consists of N parties $\mathcal{P} = \{\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_N\}$ that are called *nodes*. Each node $\mathcal{P}_i$ identifies itself through its public key pk_i for which it holds a private key sk_i that allows it to sign messages if necessary. Messages in the system are point-to-point, i.e., a node $\mathcal{P}_i$ can send a message m to another node $\mathcal{P}_j$; the node $\mathcal{P}_j$ is then convinced that the message came from $\mathcal{P}_i$ because of the signature. We assume signatures are unforgeable, so a node also cannot deny sending a particular message, since it was signed with its private key.

Network Model. **The crucial part of the model are the assumptions about the message delivery and delays. These assumptions are typically formulated by defining the abilities of an adversary, i.e., a powerful entity that watches the system and performs actions to slow it down or cause its malfunction.** The first assumption that is somewhat necessary is that the adversary cannot destroy messages that were sent, i.e., when a node sends a message, then it eventually reaches the recipient. In practice, this assumption can be enforced by sending the same message multiple times if necessary. Note also that an adversary with the ability to destroy messages would easily block every possible system, by just destroying all messages. **Given that, the most powerful ability an adversary could possibly have is**

to delay messages for an *arbitrary* amount of time. This is what we assume and what is known in the literature as the *Asynchronous Network Model*. That means the adversary can watch messages being sent and schedule their delivery in an arbitrary order.

In contrast, another popular model is the *Synchronous Network Model*², where a global-bound Δ exists such that whenever a message is sent, it is delivered after time at most Δ . As one can imagine, this assumption certainly makes protocol design easier; however, **the crucial question to address is: which of these models – asynchronous or synchronous – better fits the typical execution environment of a cryptocurrency system, i.e., the Internet.**

While the asynchronous model may seem overly conservative, since no real-life adversary has full control over the network delays, there are mechanisms that may grant him partial control, such as timed DoS attacks. Additionally, synchrony assumptions may be violated due to factors such as transient network partitions or a massive CPU load on several nodes preventing them from sending messages timely.

Finally, the key quality of any protocol meant for finance-related applications is its overall robustness, i.e., a very general notion of resilience against changing network conditions and against other unforeseeable factors. The archetypical partially synchronous algorithm PBFT [21] (and its numerous variants [1, 11, 31]) works in two modes: optimistic and pessimistic. **The often-claimed simplicity of PBFT indeed manifests itself in the optimistic mode, but the pessimistic mode (that could as well become the default one under unfavorable network conditions) is in fact a completely separate algorithm that is objectively complex and thus prone to implementation errors.** Notably, Miller *et al.* in [35] demonstrate an attack scenario on protocols from the PBFT family that completely blocks their operation. In the same paper [35], **it is also reasoned that asynchronous protocols are substantially more robust, as the model somewhat forces a single, homogeneous operation mode of the algorithm. As such, we believe that the asynchronous model, even though it enforces stricter conditions, is the best way to describe the environment in which our system is going to operate.**

Node Faults. **For the kind of system we are trying to design, one cannot assume that all the nodes always proceed according to the protocol.** A node could simply crash, or go offline, and thus stop sending messages. Alternatively, a node or a set of nodes could act maliciously (be controlled by the adversary) and send arbitrary messages in order to confuse the remaining nodes and simply break the protocol. These latter kinds of nodes are typically referred to in the literature as *dishonest*, *malicious*, *faulty*, or *Byzantine* nodes, and a protocol that solves a given problem in the presence of dishonest nodes is called Byzantine Fault Tolerant (BFT). In cryptocurrency systems the presence of dishonest nodes is more than guaranteed as there will always be parties trying to take advantage of design flaws in order to gain financial benefits. **It is known that no asynchronous system can function properly (reach consensus) in the presence of $N/3$ or more dishonest nodes [10]; thus, we make the standard assumption that the total number of nodes is $N = 3f + 1$, and f of them are dishonest.**

¹In order to make this text accessible also for readers with no background in Secure Distributed Systems, the narration of the paper focuses on providing intuitions and explaining the core ideas, at the cost of occasionally being slightly informal. At the same time, we stay mathematically rigorous when it comes to theorems and proofs.

²The Synchronous Model comes in several variants depending on whether the global bound Δ is known to the algorithm or not and whether there is an initial, finite period of asynchrony.

2.1 Our Contribution

Before presenting our contributions, let us formalize the problem of building a total ordering of transactions, which in the literature is known under the name of *Atomic Broadcast*.

DEFINITION 2.1 (ATOMIC BROADCAST). *Atomic Broadcast is a problem in which a set of nodes commonly constructs a total ordering of a set of transactions, where the transactions arrive at nodes in an on-line fashion, i.e., might not be given all at once. In a protocol that is meant to solve Atomic Broadcast, the following primitives are defined for every node:*

- (1) **Input(tx)** is called whenever a new transaction tx is received by the node.
- (2) **Output(pos, tx)** is called when the node decides to place the transaction tx at the position $pos \in \mathbb{N}$.

We say that such a protocol implements Atomic Broadcast if it meets all the requirements listed below:

- (1) **Total Order.** Every node outputs a sequence of transactions in an incremental manner, i.e., before outputting a transaction at position $pos \in \mathbb{N}$ it must have before output transactions at all positions $< pos$, and only one transaction can be output at a given position.
- (2) **Agreement.** Whenever an honest node outputs a transaction tx at position pos , every other honest node eventually outputs the same transaction at this position.
- (3) **Censorship Resilience.** Every transaction tx that is input at some honest node is eventually output by all honest nodes.

The above definition formally describes the setting in which all nodes listen for transactions and commonly construct an ordering of them. While the Total Order and Agreement properties ensure that all the honest nodes always produce the same ordering, the Censorship Resilience property is to guarantee that no transaction is lost due to censorship (especially important in financial applications) but also guarantees that the system makes progress and does not become stuck as long as new transactions are being received.

Let us briefly discuss how the performance of such an Atomic Broadcast protocol is measured. The notion of time is not so straightforward when talking about asynchronous protocols, as the adversary has the power to arbitrarily delay messages between nodes. For this reason, the running time of such a protocol is usually measured in the number of asynchronous rounds it takes to achieve a specific goal [19]. Roughly speaking, the protocol advances to round number r whenever all messages sent in round $r - 2$ have been delivered; for a detailed description of asynchronous rounds, we refer the reader to Section G.

The second performance measure that is typically used when evaluating such protocols is *communication complexity*, i.e., how much data is being sent between the nodes (on average, by a single honest node). To reduce the number of parameters required to state this result, we assume that a transaction, a digital signature, an index of a node, etc., all incur a constant communication overhead when sent; in other words, the communication complexity is measured in machine words, which are assumed to fit all the above objects. Our first contribution is the Aleph protocol for solving Atomic Broadcast whose properties are described in the following

THEOREM 2.1 (ATOMIC BROADCAST). *The Aleph protocol implements Atomic Broadcast over $N = 3f + 1$ nodes in asynchronous network, of which f are dishonest and has the following properties:*

- (1) **Latency.** For every transaction tx that is input to at least k honest nodes, the expected number of asynchronous rounds until it is output by every honest node is $O(N/k)$.
- (2) **Communication Complexity.** The total communication complexity of the protocol in R rounds³ is $O(T + R \cdot N^2 \log N)$ per node, where T is the total number of transactions input to honest nodes during R rounds.

We believe that the most important parameter of such a protocol and also the one that is hardest to optimize in practice⁴ is the transaction latency. This is why we mainly focus on achieving the optimal $O(1)$ latency⁵. In the Honey Badger BFT [35] the latency is $\Omega(\log N)$ in the optimistic case, while it becomes $\Omega(\beta \log N)$ when there are roughly $\beta N^2 \log N$ unordered transactions in the system at the time when tx is input. In contrast, the latency of our protocol is $O(1)$ independently from the load.

Similarly to [35] we need to make somewhat troublesome assumptions about the rate at which transactions arrive in the system to reason about the communication complexity, see Section 2.2 for comparison. Still, in the regime of [35] where a steady inflow of transactions is being assumed (i.e., roughly N^2 per round), we match the optimal $O(1)$ communication complexity per transaction of Honey Badger BFT [35]. As a practical note, we also mention that a variant of our protocol (Section A) achieves the optimal 3-round validation latency in the “optimistic case” akin to partially synchronous protocols from the PBFT family [11, 21] (see Section C.3 for a proof). On top of that, our protocol satisfies the so-called *Responsiveness* property [40] which intuitively means that it makes progress at speed proportional to the instantaneous network throughput and is not slowed down by predefined timeouts.

Importantly, we believe that aside from achieving optimal latency and communication complexity, Aleph is simple, clean, and easy to understand, which makes it well fit for a practical implementation. This is a consequence of its modular structure, which separates entirely the network communication layer from the protocol logic. More specifically, the only role of the network layer is to maintain a common (among nodes) data structure called a Communication History DAG, and the protocol logic is then stated entirely through combinatorial properties of this structure. We introduce our protocol in Section 3 and formally prove its properties in Sections C, D and E.

Another important property of our protocol is that unlike [16, 35], it does not require a trusted dealer. The role of a trusted dealer is typically to distribute certain cryptographic keys among nodes before the protocol starts. Clearly, in blockchain systems, no trusted entities can be assumed to exist, and thus a trusted setup is tricky if not impossible to achieve in real-world applications.

Our second contribution is an important, stand-alone component of our protocol that allows us to remove the trusted dealer assumption. More specifically, it can be seen as a protocol for generating

³Here by a round we formally mean a DAG-rounds, as formally defined in Section 3.1.

⁴The bandwidth is potentially unbounded and can be improved in various ways, while the speed of light puts a hard limit on the network delay.

⁵The $O(1)$ latency is achieved for transactions that are input to at least $k = \Omega(N)$ honest nodes. This is the same regime as in [35] where it is assumed that $k \geq 2/3N$.

unpredictable, common randomness, or in other words, it implements an ABFT Randomness Beacon. Such a source of randomness is indispensable in any ABFT protocol for Atomic Broadcast, since by the FLP-impossibility result [24], it is not possible to reach consensus in this model using a deterministic protocol. Below we give a formalization of what it means for a protocol to implement such a randomness source. The number λ that appears below is the so-called security parameter (i.e., the length of a hash, digital signature, etc.).

DEFINITION 2.2 (RANDOMNESS BEACON). *We say that a pair of protocols (Setup, Toss(m)) implements a Randomness Beacon if after running Setup once, each execution of Toss(m) (for any nonce $m \in \{0, 1\}^*$) results in λ fresh random bits. More formally, we require*

- **Termination.** *All honest nodes correctly terminate after running either Setup or Toss(m),*
- **Correctness.** *For a nonce $m \in \{0, 1\}^*$, Toss(m) results in all honest nodes outputting a common bitstring $\sigma_m \in \{0, 1\}^\lambda$,*
- **Unpredictability.** *No computationally bounded adversary can predict the outcome of Toss(m) with non-negligible probability.*

In the literature such a source of randomness (especially the variant that outputs just a single bit) is often called a *Common Coin* [15, 16, 35] or a *Global Coin* [19]. We note that once the Toss(m) protocol terminates, the value of σ_m is known, and after revealing it, another execution of Toss(m) will not provide new random bits; thus, the Unpredictability property is meant to be satisfied only before Toss(m) is initiated. As our second main contribution, in Section 4 we introduce the ABFT-Beacon protocol, and in Section E we prove the following

THEOREM 2.2 (ABFT-Beacon). *The ABFT-Beacon protocol implements a Randomness Beacon (Setup, Toss(m)) such that:*

- *the Setup phase takes $O(1)$ asynchronous rounds to complete and has $O(N^2 \log N)$ communication complexity per node,*
- *each subsequent call to Toss(m) takes 1 asynchronous round and has $O(N)$ communication complexity per node.*

We also remark that the ABFT-Beacon is relatively light when it comes to computational complexity, as the setup requires roughly $O(N^3)$ time per node, and each subsequent toss takes typically $O(N)$ time (under a slightly relaxed adversarial setting); see Section 4.

As an addition to our positive results, in Section H we introduce the *Fork Bomb* – a spam attack scenario that affects most known DAG-based protocols. In this attack, malicious nodes force honest nodes to download exponential amounts of data and thus likely crash their machines. This attack when attempted to prevent at the implementation layer by banning “suspect nodes” is likely to harm honest nodes as well. Thus, we strongly believe that without a mechanism preventing this kind of attacks already at the protocol layer, liveness is not guaranteed. The basic version of Aleph is resistant against this attack through the use of reliable broadcast to disseminate information among nodes. In Section A we also show a mechanism to defend against this attack for a gossip-based variant of Aleph.

2.2 Related Work

Atomic Broadcast. For an excellent introduction to the field of Distributed Computing and overview of Atomic Broadcast and Consensus protocols we refer the reader to the book [13]. A more recent work of [18] surveys existing consensus protocols in the context of cryptocurrency systems.

The line of work on synchronous BFT protocols was initiated in [21] with the introduction of PBFT. PBFT and its numerous variants [1, 11, 31] tolerate byzantine faults, yet their efficiency relies on the good behavior of the network, and drops significantly when entering the (in some cases implicit) “pessimistic mode”. As thoroughly reasoned in [35], synchronous algorithms might not be well suited for blockchain-related applications, because of their lack of robustness and vulnerability to certain types of attacks.

In the realm of asynchronous BFT protocols, a large part of the literature focuses on the more classical Consensus problem, i.e., reaching binary agreement by all the honest nodes. As one can imagine, ordering transactions can be reduced to a sequence of binary decisions, and indeed there are known generic reductions that solve Atomic Broadcast by running Consensus multiple times [5, 15, 22, 36]. However, all these reductions either increase the number of rounds by a super-constant factor or introduce a significant communication overhead. Thus, even though Consensus protocols with optimal number of $O(1)$ rounds [19] and optimal communication complexity [16, 38] were known early on, only in the recent work of [35] the Honey Badger BFT (HBBFT) protocol with optimal $O(1)$ communication complexity per transaction was proposed.

The comparison of our protocol to HBBFT is not straightforward since the models of transaction arrivals differ. Roughly, HBBFT assumes that at every round, every node has $\Omega(N^2)$ transactions in its buffer. Under this assumption the communication complexity of HBBFT per epoch, per node is roughly $O(N^2)$, and also $\Omega(N^2)$ transactions are ordered in one epoch, hence the optimal $O(1)$ per transaction is achieved. However, the design of HBBFT that is optimized towards low communication complexity has the negative effect that the latency might be large under high load. More precisely, if βN^2 transactions are pending in the system⁶ when tx is being input, the latency of tx is $\approx \beta$ epochs, thus $\approx \beta \log(N)$ rounds. Our algorithm, when adjusted to this model would achieve $\approx \beta$ rounds of latency (thus $\log(N)$ -factor improvement), while retaining the same, optimal communication complexity.

In this paper we propose a different assumption on the transaction buffers that allows us to better demonstrate the capabilities of our protocol when it comes to latency. We assume that at every round the ratio between lengths of transaction buffers of any two honest nodes is at most a fixed constant. In this model, our protocol achieves $O(1)$ latency, while a natural adaptation of HBBFT would achieve latency $O(\log(N))$, thus again a factor- $\log(N)$ improvement. A qualitative improvement over HBBFT that we achieve in this paper is that we completely get rid of the trusted dealer assumption. We also note that the definitions of Atomic Broadcast slightly differ between this paper and [35]: we achieve Censorship Resilience

⁶For the sake of this comparison we only consider transactions that have been input to $\Omega(N)$ honest nodes.

assuming that it was input to even a single honest node, while in [35] it has to be input to $\Omega(N)$ nodes.

The recent work⁷ of Abraham et al. [3] studies a closely related Validated Asynchronous Byzantine Agreement (VABA) problem, which is, roughly speaking, the problem of picking one value out of N proposed by the nodes. The protocol in [3] achieves $O(1)$ latency and has optimal communication complexity of $O(N)$ per node. We believe that combining it with the ideas present in HoneyBadgerBFT can yield an algorithm with the same communication complexity as HoneyBadgerBFT but with latency improved by a factor of $\log N$. However, when compared to ours, such a protocol still requires a trusted dealer and achieves weaker censorship resilience.

Finally, we remark that our algorithm is based on maintaining a DAG data structure representing the communication history of the protocol. This can be seen as a realization of Lamport’s “happened-before” relation [33] or the so-called Causal Order [27]. To the best of our knowledge, the first instance of using DAGs to design asynchronous protocols is the work of [37]. More recently DAGs gained more attention in the blockchain space [4, 23, 44].

Common Randomness. For a thorough overview of previous work on generating randomness in distributed systems and a discussion on the novelty of our solution we refer to Section 4.1.

3 ATOMIC BROADCAST

This section is devoted to an outline and discussion of the Aleph protocol. We start by sketching the overall idea of the algorithm and explaining how it works from a high-level perspective. In this process we present the most basic variant of the Aleph protocol, which already contains all the crucial ideas and gives a provably correct implementation of Atomic Broadcast. On the theoretical side, however, this basic variant might suffer from a slightly sub-optimal communication complexity. In Section D we describe a simple tweak to the protocol which allows us to finally achieve the communication complexity as claimed in Theorem 2.1. We refer to Sections C and D for proofs of correctness and efficiency of Aleph.

3.1 Asynchronous communication as a DAG

The concept of a “communication round” as explained in the preliminaries, is rather natural in the synchronous model, but might be hard to grasp when talking about asynchronous settings. This is one of the reasons why asynchronous models are, generally speaking, harder to work with than their (partially) synchronous counterparts, especially when it comes to proving properties of such protocols.

Units and DAGs. To overcome the above issue, we present a general framework for constructing and analyzing asynchronous protocols that is based on maintaining (by all the nodes) the common “communication history” in the form of an append-only structure: a DAG (Directed Acyclic Graph).

The DAGs that we consider in this paper originate from the following idea: we would like to divide the execution of the algorithm into virtual rounds so that in every round r every node emits exactly one Unit that should be thought of as a message broadcast to all the other nodes. Moreover, every such unit should have “pointers” to a

large enough number of units from the previous round, emitted by other nodes. Such pointers can be realized by including hashes of the corresponding units, which, assuming that our hash function is collision-free, allows to uniquely determine the “parent units”. Formally, every unit has the following fields:

- **Creator.** Index and a signature of unit’s creator.
- **Parents.** A list of units’ hashes.
- **Data.** Additional data to be included in the unit.

The fact that a unit U has another unit V included as its parent signifies that the information piece carried by V was known to U ’s creator at the time of constructing U , i.e., V causally precedes U . All the nodes are expected to generate such units in accordance to some initially agreed upon rules (defined by the protocol) and maintain their local copies of the common DAG, to which new units are continuously being added.

Communication History DAGs. To define the basic rules of creating units note first that this DAG structure induces a partial ordering on the set of units. To emphasize this fact, we often write $V \leq U$ if either V is a parent of U , or more generally (transitive closure) that V can be reached from U by taking the “parent pointer” several times. This also gives rise to the notion of DAG-round of a unit U that is defined to be the maximum length of a downward chain starting at U . In other words, recursively, a unit with no parents has DAG-round 0 and otherwise a unit has DAG-round equal to the maximum of DAG-rounds of its parents plus one. We denote the DAG-round of a unit U by $R(U)$. Usually we just write “round” instead of DAG-round except for parts where the distinction between DAG-round and async-round (as defined in Section G) is relevant (i.e., mostly Section D). We now proceed to define the notion of a ch-DAG (communication history DAG) that serves as a backbone of the Aleph protocol.

DEFINITION 3.1 (ch-DAG). We say that a set of units $\mathcal{D}$ created by $N = 3f + 1$ nodes forms a ch-DAG if the parents of every unit in $\mathcal{D}$ also belong to $\mathcal{D}$ and additionally the following conditions hold true

- (1) **Chains.** For each honest node $\mathcal{P}_i \in \mathcal{P}$, the set of units in $\mathcal{D}$ created by $\mathcal{P}_i$ forms a chain.
- (2) **Dissemination.** Every round- r unit in $\mathcal{D}$ has at least $2f + 1$ parents of round $r - 1$.
- (3) **Diversity** Every unit in $\mathcal{D}$ has parents created by pairwise distinct nodes.

What the above definition tries to achieve is that, roughly, every node should create one unit in every round and it should do so after learning a large enough portion (i.e. at least $2f + 1$) of units created in the previous round. The purpose of the **Chains** rule is to forbid forking, i.e., a situation where a node creates more than one unit in a single round. The second rule, **Dissemination**, guarantees that a node creating a unit in round r learned as much as possible from the previous round – note that as there are only $N - f = 2f + 1$ honest nodes in the system, we cannot require that a node receives more than $2f + 1$ units, as byzantine nodes might not have created them. The unit may have additional parents, but the **Diversity** rule ensures that they are created by different nodes – otherwise units could become progressively bigger as the ch-DAG grows, by linking

⁷We would like to thank the anonymous reviewer for bringing this work to our attention.

to all the units in existence, hence increasing the communication complexity of the protocol.

Building DAGs. The pseudocode DAG – Grow($\mathcal{D}$) provides a basic implementation of a node that takes part in maintaining a common DAG. Such a node initializes first $\mathcal{D}$ to an empty DAG and then runs two procedures CreateUnit and ReceiveUnits in parallel. To create a unit at round r , we simply wait until $2f + 1$ units of round $r - 1$ are available and then, for every node, we pick its unit of highest round (i.e., of round at most $r - 1$) and include all these N (unless some nodes created no units at all, in which case $< N$) units as parents of the unit. In other words, we wait just enough until we can advance to the next round, and attach to our round- r unit everything we knew at that point in time.

DAG-Grow($\mathcal{D}$):

```

1 CreateUnit(data):
2   for  $r = 0, 1, 2, \dots$  do
3     if  $r > 0$  then
4       wait until  $|\{U \in \mathcal{D} : R(U) = r - 1\}| \geq 2f + 1$ 
5        $P \leftarrow \{\text{maximal } \mathcal{P}_i\text{'s unit of round } < r \text{ in } \mathcal{D} : \mathcal{P}_i \in \mathcal{P}\}$ 
6       create a new unit  $U$  with  $P$  as parents
7       include data in  $U$ 
8       add  $U$  to  $\mathcal{D}$ 
9       RBC( $U$ )
10 ReceiveUnits:
11   loop forever
12     upon receiving a unit  $U$  via RBC do
13       add  $U$  to  $\mathcal{D}$ 

```

Both CreateUnit and ReceiveUnits make use of a primitive called RBC that stands for *Reliable Broadcast*. This is an asynchronous protocol that guarantees that every unit broadcast by an honest node is eventually received by all honest nodes. More specifically we use the validated RBC protocol which also ensures that incorrect units (with incorrect signatures or incorrect number of parents, etc.) are never broadcast successfully. Furthermore, our version of RBC forces every node to broadcast exactly one unit per round, thus effectively banning forks. We refer to Section F for a thorough discussion on Reliable Broadcast.

The validated RBC algorithm internally checks whether a certain $\text{Valid}(U)$ predicate is satisfied when receiving a unit U . This predicate makes sure that the requirements in Definition 3.1 are satisfied, as well as verifies that certain data, required by the protocol is included in U . Consequently, only valid units are added to the local ch-DAGs maintained by nodes. Furthermore, as alluded above, there can be only a single copy of a unit created by a particular node in a particular round. This guarantees that the local copies of ch-DAGs maintained by different nodes always stay consistent.

Let us now describe more formally the desired properties of a protocol used to grow and maintain a common ch-DAG. For this it is useful to introduce the following convention: we denote the local copy of the ch-DAG maintained by the i th node by $\mathcal{D}_i$.

DEFINITION 3.2. *We distinguish the following properties of a protocol for constructing a common ch-DAG*

- (1) **Reliable:** *for every unit U added to a local copy $\mathcal{D}_i$ of an honest node $\mathcal{P}_i$, U is eventually added to the local copy $\mathcal{D}_j$ of every honest node $\mathcal{P}_j$.*
- (2) **Ever-expanding:** *for every honest node $\mathcal{P}_i$ the local copy $\mathcal{D}_i$ grows indefinitely, i.e., $R(\mathcal{D}_i) := \max\{r(U) \mid U \in \mathcal{D}_i\}$ is unbounded.*
- (3) **Fork-free:** *whenever two honest nodes i_1, i_2 hold units $U_1 \in \mathcal{D}_{i_1}$ and $U_2 \in \mathcal{D}_{i_2}$ such that both U_1, U_2 have the same creator and the same round number, then $U_1 = U_2$.*

Having these properties defined, we are ready to state the main theorem describing the process of constructing ch-DAG by the DAG-Grow protocol.

THEOREM 3.1. *The DAG-Grow protocol is reliable, ever-expanding, and fork-free. Additionally, during asynchronous round r each honest node holds a local copy of $\mathcal{D}$ of round at least $\Omega(r)$.*

For a proof we refer the reader to Section B.1.

Benefits of Using DAGs. After formally introducing the idea of a ch-DAG and explaining how it is constructed we are finally ready to discuss the significance of this concept. First of all, ch-DAGs allow for a clean and conceptually simple separation between the communication layer (sending and receiving messages between nodes) and the protocol logic (mainly deciding on relative order of transactions). Specifically, the network layer is simply realized by running Reliable Broadcast in the background, and the protocol logic is implemented as running off-line computations on the local copy of the ch-DAG. One can think of the local copy of the ch-DAG as the *state* of the corresponding node; all decisions of a node are based solely on its state. One important consequence of this separation is that the network layer, being independent from the logic, can be as well implemented differently, for instance using ordinary broadcast or random gossip (see Section A).

In the protocol based on ch-DAGs the concept of an adversary and his capabilities are arguably easier to understand. The ability of the adversary to delay a message now translates into a unit being added to some node's local copy of the ch-DAG with a delay. Nonetheless, every unit that has ever been created will still be eventually added to all the ch-DAGs maintained by honest nodes. A consequence of the above is that the adversary can (almost arbitrarily) manipulate the structure of the ch-DAG, or, in other words, he is able to force a given set of round- $(r - 1)$ parents for a given round- r unit. But even then, it needs to pick at least $2f + 1$ round- $(r - 1)$ units, which enforces that more than a half of every unit's parents are created by honest nodes.

3.2 Atomic broadcast via ch-DAG

In this section we show how to build an Atomic Broadcast protocol based on the ch-DAG maintained locally by all the nodes. Recall that nodes receive transactions in an on-line fashion and their goal is to construct a common linear ordering of these transactions. Every node thus gathers transactions in its local buffer and whenever it creates a new unit, all transactions from its buffer are included in the data field of the new unit and removed from the buffer.⁸

⁸This is the simplest possible strategy for including transactions in the ch-DAG and while it is provably correct it may not be optimal in terms of communication complexity. We show how to fix this in Section D.

Thus, to construct a common linear ordering on transactions it suffices to construct a linear ordering of units in the ch-DAG (the transactions within units can be ordered in an arbitrary yet fixed manner, for instance alphabetically). The ordering that we are going to construct also has the nice property that it *extends* the ordering of units induced by the ch-DAG (i.e. the causal order).

Let us remark at this point that all primitives that we describe in this section take a local copy $\mathcal{D}$ of the ch-DAG as one of their parameters and return either

- a result (which might be a single bit, a unit, etc.), or
- $\perp$, signifying that the result is not yet available in $\mathcal{D}$.

The latter means that in order to read off the result, the local copy $\mathcal{D}$ needs to grow further. We occasionally omit the $\mathcal{D}$ argument, when it is clear from the context which local copy should be used.

Ordering Units. The main primitive that is used to order units in the ch-DAG, `OrderUnits`, takes a local copy $\mathcal{D}$ of the ch-DAG and outputs a list `linord` that contains a subset of units in $\mathcal{D}$. This list is a prefix of the global linear ordering that is commonly generated by all the nodes. We note that `linord` will normally not contain all the units in $\mathcal{D}$ but a certain subset of them. More precisely, `linord` contains all units in $\mathcal{D}$ except those created in the most recent (typically around 3) rounds. While these top units cannot be ordered yet, the structural information about the ch-DAG they carry is used to order the units below them. Intuitively, the algorithm that is run in the ch-DAG at round r makes decisions regarding units that are several rounds deeper, thus the delay.

Note that different nodes might hold different versions of the ch-DAG at any specific point in time, but what we guarantee in the ch-DAG growing protocol is that all copies of ch-DAG are consistent, i.e., all the honest nodes always see exactly the same version of every unit ever created, and that every unit is eventually received by all honest nodes. The function `OrderUnits` is designed in such a way that even when called on different versions of the ch-DAG $\mathcal{D}_1$, $\mathcal{D}_2$, as long as they are consistent, the respective outputs `linorder1`, `linorder2` also agree, i.e., one of them is a prefix of the other.

The `OrderUnits` primitive is rather straightforward. At every round r , one unit V_r from among units of round r is chosen to be a “head” of this round, as implemented in `ChooseHead`. Next, all the units in $\mathcal{D}$ that are less than V_r , but are not less than any of $V_0, V_1, \dots, V_{r-1}$ form the r th batch of units. The batches are sorted by their round numbers and units within batches are sorted topologically breaking ties using the units’ hashes.

Choosing Heads. Perhaps surprisingly, the only nontrivial part of the protocol is choosing a head unit for each round. It is not hard to see that simple strategies for choosing a head fail in an asynchronous network. For instance, one could try picking always the unit created by the first node to be the head: this does not work because the first node might be byzantine and never create any unit. To get around this issue, one could try another tempting strategy: to choose a head for round r , every node waits till round $r + 10$, and declares as the head the unit of round r in its copy of the ch-DAG that has the smallest hash. This strategy is also doomed to fail, as it might cause inconsistent choices of heads between nodes: this can happen when some of the nodes see a unit with a very small hash in their ch-DAG while the remaining ones did not receive it yet, which might have either happened just by accident or was

Aleph:

```

1 OrderUnits( $\mathcal{D}$ ):
2   linord  $\leftarrow$  []
3   for  $r = 0, 1, \dots, R(\mathcal{D})$  do
4      $V_r \leftarrow \text{ChooseHead}(r, \mathcal{D})$ 
5     if  $V_r = \perp$  then break
6     batch  $\leftarrow \{U \in \mathcal{D} : U \leq V_r \text{ and } U \notin \text{linord}\}$ 
7     order batch deterministically
8     append batch to linord
9   output linord
10 ChooseHead( $r, \mathcal{D}$ ):
11    $\pi_r \leftarrow \text{GeneratePermutation}(r, \mathcal{D})$ 
12   if  $\pi_r = \perp$  then output  $\perp$ 
13   else
14      $(U_1, U_2, \dots, U_k) \leftarrow \pi_r$ 
15     for  $i = 1, 2, \dots, k$  do
16       if  $\text{Decide}(U_i, \mathcal{D}) = 1$  then
17         output  $U_i$ 
18       else if  $\text{Decide}(U_i, \mathcal{D}) = \perp$  then
19         output  $\perp$ 
20   output  $\perp$ 

```

forced by actions of the adversary. Note that under asynchrony, one can never be sure whether missing a unit from some rounds back means that there is a huge delay in receiving it or it was never created (the creator is byzantine). More generally, this also justifies that in any asynchronous BFT protocol it is never correct to wait for one fixed node to send a particular message.

Our strategy for choosing a head in round r is quite simple: pick the first unit (i.e., with lowest creator id) that is visible by every node. The obvious gap in the above is how do we decide that a particular unit is visible? As observed in the example above, waiting a fixed number of rounds is not sufficient, as seeing a unit locally does not imply that all other nodes see it as well. Instead, we need to solve an instance of *Binary Consensus* (also called *Binary Agreement*). In the pseudocode this is represented by a `Decide( $U$ )` function that outputs 0 or 1; we discuss it in the subsequent paragraph.

There is another minor adjustment to the above scheme that aims at decreasing the worst case latency, which in the just introduced version is $O(\log N)$ rounds⁹. When using a random permutation (unpredictable by the adversary) instead of the order given by the units creator indices, the latency provably goes down to $O(1)$. Such an unpredictable random permutation is realized by the `GeneratePermutation` function.

Consensus. For a round- $(r + 1)$ unit U , by $\downarrow(U)$ we denote the set of all round- r parents of U . Consider now a unit U_0 in round r ; we would like the result of `Decide( $U_0, \mathcal{D}$ )` to “answer” the question whether all nodes can see the unit U_0 . This is done through voting: starting from round $r + 1$ every unit casts a “virtual” vote¹⁰ on U_0 . These votes are called virtual because they are never really

⁹The reason is that the adversary could cause the first $\Omega(N)$ units to be decided 0. Since the delay of each such decision is a geometric random variable with expectation $\Theta(1)$, the maximum of $\Omega(N)$ of them is $\Omega(\log N)$.

¹⁰The idea of virtual voting was used for the first time in [37].

broadcast to other nodes, but they are computed from the ch-DAG. For instance, at round $r + 1$, a unit U is said to vote 1 if $U_0 < U$ and 0 otherwise, which directly corresponds to the intuition that the nodes are trying to figure out whether U_0 is visible or not.

Aleph-Consensus($\mathcal{D}$):

```

1  Vote( $U_0, U, \mathcal{D}$ ):
2  | if  $R(U) \leq R(U_0) + 1$  then output11 [ $U_0 < U$ ]
3  | else
4  |    $A \leftarrow \{\text{Vote}(U_0, V, \mathcal{D}) : V \in \downarrow(U)\}$ 
5  |   if  $A = \{\sigma\}$  then output  $\sigma$ 
6  |   else output CommonVote( $U_0, R(U), \mathcal{D}$ )
7  UnitDecide( $U_0, U, \mathcal{D}$ ):
8  | if  $R(U) < R(U_0) + 2$  then output  $\perp$ 
9  |  $v \leftarrow \text{CommonVote}(U_0, U)$ 
10 | if  $|\{V \in \downarrow(U) : \text{Vote}(U_0, V) = v\}| \geq 2f + 1$  then output  $v$ 
11 | else output  $\perp$ 
12 Decide( $U_0, \mathcal{D}$ ):
13 | if  $\exists U \in \mathcal{D} \text{ UnitDecide}(U_0, U, \mathcal{D}) = \sigma \in \{0, 1\}$  then
14 |   output  $\sigma$ 
15 | else output  $\perp$ 

```

Starting from round $r + 2$ every unit can either make a final decision on a unit or simply vote again. This process is guided by the function $\text{CommonVote}(U_0, r', \mathcal{D})$ that provides a common bit $\in \{0, 1\}$ for every round $r' \geq r + 2$. Suppose now that U is of round $r' \geq r + 2$ and at least $2f + 1$ of its round- $(r' - 1)$ parents in the ch-DAG voted 1 for U_0 , then if $\text{CommonVote}(U_0, r', \mathcal{D}) = 1$, then unit U is declared to decide U_0 as 1. Otherwise, if either there is no supermajority vote among parents (i.e., at least $2f + 1$ matching votes) or the supermajority vote does not agree with the CommonVote for this round, the decision is not made yet. In this case, the unit U revotes using either the vote suggested by its parents (in case it was unanimous) or using the default CommonVote. Whenever any of the units $U \in \mathcal{D}$ decides U_0 then it is considered decided with that particular decision bit.

Crucially, the process is designed in such a way that when some unit U decides $\sigma \in \{0, 1\}$ on some unit U_0 then we prove that no unit ever decides $\bar{\sigma}$ (the negation of σ) on U_0 and also that every unit of high enough round decides σ on U_0 as well. At this point it is already not hard to see that if a unit U makes decision σ on U_0 then it follows that **all** the units of round $R(U)$ (and any round higher than that) vote σ on U_0 . To prove that observe that if a unit V of round r votes σ' then either:

- all its parents voted σ' and hence $\sigma' = \sigma$, because U and V have at least $f + 1$ parents in common, or
- $\sigma' = \text{CommonVote}(U_0, R(V), \mathcal{D})$, but since U decided σ for U_0 then $\text{CommonVote}(U_0, R(U), \mathcal{D}) = \sigma$ and thus $\sigma = \sigma'$ because $R(U) = R(V)$.

The above gives a sketch of “safety” proof of the protocol, i.e., that there will never be inconsistent decisions regarding a unit U_0 .

¹¹In the expression $[U_0 < U]$ we use the Iverson bracket notation, i.e., $[U_0 < U] = 1$ if $U_0 < U$ and it is 0 otherwise.

Another property that is desirable for a consensus protocol is that it always terminates, i.e., eventually outputs a consistent decision. In the view of the FLP Theorem [24] this cannot be achieved in the absence of randomness in the protocol. This is the reason why we need to inject random bits to the protocol and this is done in CommonVote. We show that, roughly, if CommonVote provides a random bit (that cannot be predicted by the adversary well in advance) then at every round the decision process terminates with probability at least $1/2$. Thus, the expected number of rounds until termination is $O(1)$.

We provide formal proofs of properties of the above protocol in Section D. In the next section, we show how the randomness in the protocol is generated to implement CommonVote.

3.3 Common Randomness

As already alluded to in Subsection 3.2, due to FLP-impossibility [24] there is no way to achieve binary agreement in finite number of rounds when no randomness is present in the protocol. We also note here that not any kind of randomness will do and the mere fact that a protocol is randomized does not “protect” it from FLP-impossibility. It is crucial to keep the random bits hidden from the adversary till a certain point, intuitively until the adversary has committed to what decision to pursue for a given unit at a certain round. When the random bit is revealed after this commitment, then there is a probability of $1/2$ that the adversary “is caught” and cannot delay the consensus decision further.

That means, in particular, that a source of randomness where the nodes are initialized with a common random seed and use a pseudo-random number generator to extract fresh bits is not sufficient, since such a strategy actually makes the algorithm deterministic.

Another issue when generating randomness for the protocol is that it should be *common*, thus in other words, the random bit output by every honest node in a particular round r should agree between nodes. While this is strictly required for safety of the algorithm that is presented in the paragraph on Consensus, one can also consider simple variants thereof for which a relaxed version of commonness is sufficient. More specifically, if the bit output at any round is common with probability $p \in (0, 1]$ then one can achieve consensus in $O(1/p)$ rounds. Indeed, already in [9] Bracha constructed such a protocol and observed that if every node locally tosses a coin independently from the others than this becomes a randomness source that is common with probability $p \approx 2^{-N}$ and thus gives a protocol that takes $O(2^N)$ rounds to terminate. We refer to Subsection 2.2 for an overview of previous work on common randomness.

In our protocol, randomness is injected via a single procedure SecretBits whose properties we formalize in the definition below

DEFINITION 3.3 (SECRET BITS). *The SecretBits($i, \text{revealRound}$) primitive takes an index $i \in \{1, 2, \dots, N\}$ and $\text{revealRound} \in \mathbb{N}$ as parameters, outputs a λ -bits secret s , and has the following properties whenever initiated by the ChooseHead protocol*

- (1) *no computationally bounded adversary can guess s with non-negligible probability, as long as no honest node has yet created a unit of round revealRound ,*
- (2) *the secret s can be extracted by every node that holds any unit of round $\text{revealRound} + 1$.*

As one might observe, the first parameter i of the SecretBits function seems redundant. Indeed, given an implementation of SecretBits, we could as well use SecretBits(1, ·) in place of SecretBits(i , ·) for any $i \in [N]$ (here and in the remaining part of the paper $[N]$ stands for $\{1, \dots, N\}$) and it would seemingly still satisfy the definition above. The issue here is very subtle and will become clear only in Section 4, where we construct a SecretBits function whose computability is somewhat sensitive to what i it is run with¹². In fact, we recommend the reader to ignore this auxiliary parameter, as our main implementation of SecretBits(i , *revealRound*) is anyway oblivious to the value of i and thus essentially there is exactly one secret per round in the protocol.

The simplest attempt to implement SecretBits would be perhaps to use an external trusted party that observes the growth of the ch-DAG and emits one secret per round, whenever the time comes. Clearly, the correctness of the above relies crucially on the dealer being honest, which is an assumption we cannot make: indeed if there was such an honest entity, why do not we just let him order transactions instead of designing such a complicated protocol?

Instead, our approach is to utilize a threshold secret scheme (see [45]). In short, at round r every node is instructed to include its *share* of the secret (that is hidden from every node) in the unit it creates. Then, in round $r + 1$, every node collects $f + 1$ different such shares included in the previous round and reconstructs the secret from them. Crucially, any set of f or less shares is not enough to derive the secret, thus the adversary controlling f nodes still needs at least one share from an honest node. While this allows the adversary to extract the secret one round earlier than the honest nodes, this advantage turns out to be irrelevant, as, intuitively, he needed to commit to certain actions several turns in advance (see Section C for a detailed analysis).

Given the SecretBits primitive, we are ready to implement GeneratePermutation and CommonVote (the pseudocode is given in the table Aleph-CommonRandomness).

In the pseudocode, by hash we denote a hash function¹³ that takes an input and outputs a bistring of length λ . We remark that the CommonVote at round $R(U_0) + 4$ being 0 is to enforce that units that are invisible at this round will be quickly decided negatively (see Lemma C.4). To explain the intuition behind GeneratePermutation, suppose for a second that SecretBits(i , $r + 4$) outputs the same secret x independently of i (as it is the case for our main algorithm). Then the above pseudocode assigns a pseudorandom priority $\text{hash}(x||U)$ (where, for brevity U denotes some serialization of U) to every unit U or round r which results in a random ordering of these units, as required.

At this point, the only remaining piece of the protocol is the SecretBits procedure. We provide two implementations in the subsequent Section (see Lemma 4.1): one simple, but requiring a trusted dealer, and the second, more involved but completely trustless.

4 RANDOMNESS BEACON

The goal of this section is to construct an ABFT Randomness Beacon, in order to provide an efficient implementation of SecretBits that is

¹²More precisely, intuitively SecretBits(i , ·) is not expected to work properly if for instance node $\mathcal{P}_i$ has produced no unit at all. Also, importantly, the ChooseHead algorithm will never call SecretBits(i , ·) in such a case.

¹³We work in the standard Random Oracle Model.

Aleph-CommonRandomness:

```

1 CommonVote( $U_0, r, \mathcal{D}$ ):
2   if  $r \leq R(U_0) + 3$  then output 1
3   if  $r = R(U_0) + 4$  then output 0
4   else
5      $i \leftarrow$  the creator of  $U_0$ 
6      $x \leftarrow \text{SecretBits}(i, r, \mathcal{D})$ 
7     if  $x = \perp$  then output  $\perp$ 
8     output the first bit of  $\text{hash}(x)$ 
9 GeneratePermutation( $r, \mathcal{D}$ ):
10  for each unit  $U$  of round  $r$  in  $\mathcal{D}$  do
11     $i \leftarrow$  the creator of  $U$ 
12     $x \leftarrow \text{SecretBits}(i, r + 4, \mathcal{D})$ 
13    if  $x = \perp$  then output  $\perp$ 
14    assign priority( $U$ )  $\leftarrow \text{hash}(x||U) \in \{0, 1\}^\lambda$ 
15  let  $(U_1, U_2, \dots, U_k)$  be the units in  $\mathcal{D}$  of round  $r$  sorted by
    priority(·)
16  output  $(U_1, U_2, \dots, U_k)$ 

```

required by our Atomic Broadcast protocol. We start by a detailed review of previous works and how do they compare to the result of this paper. Subsequently we describe how to extract randomness from threshold signatures, which is a basic building block in our approach, and after that we proceed with the description of our randomness beacon.

4.1 Comparison to Previous Work on Distributed Randomness Beacons

We distinguish two main approaches for solving the problem of generating randomness in distributed systems in the presence of byzantine nodes: using Verifiable Secret Sharing (VSS) and via Threshold Signatures. We proceed to reviewing these two approaches and subsequently explain what does our work bring to the field. This discussion is succinctly summarized in Table 1.

Verifiable Secret Sharing. The rough idea of VSS can be explained as follows: a dealer first initializes a protocol to distribute shares $(x_1, x_2, \dots, x_N)$ of a secret x to the nodes $1, 2, \dots, N$ so that afterwards every set of $(f + 1)$ nodes can extract x by combining their shares, but any subset of f nodes cannot do that¹⁴. An honest dealer is requested to pick x uniformly at random (and erase x from memory); yet as one can imagine, this alone is not enough to build a reliable, distributed randomness source, with the main issue being: how to elect a dealer with the guarantee that he is honest? In the seminal paper [19] Canetti and Rabin introduce the following trick: let all nodes in the network act as dealers and perform VSS with the intention to combine all these secrets into one (say, by xoring them) that would be random and unpredictable. Turning this idea into an actual protocol that works under full asynchrony is especially tricky, yet [19] manage to do that and end up with a protocol that has $O(N^7)$ communication complexity. This $O(N^7)$ essentially comes from executing an $O(N^5)$ communication VSS protocol N^2 times. Later on this has been improved by Cachin et

¹⁴The thresholds f and $(f + 1)$ here are not the only possible, but are most relevant for our setting, see [14] for a more detailed treatment.

al. [14] to $O(N^4)$ by making the VSS protocol more efficient. The issue with the AVSS approach is that the communication cost of generating one portion of randomness is rather high – it requires to start everything from scratch when new random bits are requested. This issue is not present in the approach based on threshold signatures, where a one-time setup is run and then multiple portions of random bits can be generated cheaply.

Threshold Signatures. For a technical introduction to this approach we refer to Section 4.2. The main idea is that if the nodes hold keys for threshold signatures with threshold $f + 1$, then the threshold signature $\sigma_m \in \{0, 1\}^\lambda$ of a message m is unpredictable for the adversary and provides us with roughly λ bits of entropy.

The idea of using threshold signatures as a source of randomness is not new and has been studied in the literature [16], especially in the context of blockchain [28, 35]. While this technique indeed produces unbiased randomness **assuming** that the tossing keys have been correctly dealt to all nodes, the true challenge becomes: *How to deal secret keys without involving a trusted dealer?* This problem is known in the literature under the name of Distributed Key Generation (DKG). There has been a lot of prior work on DKG [25, 26, 29, 41], however none of the so far proposed protocols has been designed to run under full asynchrony.

Historically, the first implementation of DKG was proposed in the work of Pedersen [41]. The network model is not formally specified in [41], yet one can implement it in the synchronous BFT model with f out of $N = 3f + 1$ nodes being malicious. Pedersen’s protocol, as later shown by Gennaro et al. in [26], does not guarantee a uniform distribution over private keys; in the same work [26] a fix is proposed that closes this gap, but also in a later work [25] by the same set of authors it is shown that Pedersen’s protocol is still secure¹⁵. The DFINITY randomness beacon [28] runs DKG as setup and subsequently uses threshold signatures to generate randomness.

We note that the AVSS scheme by Cachin et al. [14] can be also used as a basis of DKG since it is polynomial-based and has the additional property of every node learning a certain “commitment” to the secret shares obtained by all the nodes, which in the setting of threshold signatures corresponds to *public keys* of the threshold scheme (see Section 4.2). This allows to construct a DKG protocol as follows: each node runs its own instance of AVSS and subsequently the nodes agree on a subset of $\geq f + 1$ such dealt secrets to build the keys for threshold signatures. This scheme looks fairly simple, yet it is extremely tricky to implement correctly. One issue is security, i.e., making sure that the adversary is not “front-running” and cannot reveal any secret too early, but more fundamentally: this algorithm requires **consensus** to choose a subset of secrets. ABFT consensus requires **randomness** (by FLP theorem [24]), which we are trying to obtain, and thus we fall into an infinite loop. To obtain consensus without the help of a trusted dealer one is forced to use variants of the Canetti-Rabin protocol [19] (recall that we are working here under the hard constraint of having no trusted dealer, which is not satisfied by most protocols in the literature) that unfortunately has very high communication complexity. Nevertheless using this idea, one can obtain an ABFT DKG protocol with $O(N^4)$ communication

¹⁵We note that our protocol can be seen as an adaptation of Pedersen’s and thus also requires an ad-hoc proof of security (provided in Lemma E.4), as the distribution of secret keys might be skewed.

Protocol	Model	Communication	
		Setup	Query
This Work	async. BFT	$O(N^2 \log N)$	$O(N)$
Canetti, Rabin [19]	async. BFT	-	$O(N^7)$
Cachin et al. [14]	async. BFT	-	$O(N^4)$
Kate et al. DKG [29]	w. sync. BFT	$O(N^3)$	-
Pedersen DKG [25, 41]	sync. BFT	$O(N^2)$	-
Gennaro et al. DKG [26]	sync. BFT	$O(N^2)$	-
DFINITY [28]	sync. BFT	$O(N^2)$	$O(N)$
RandShare [46]	sync. BFT	-	$O(N^2)$
SCRAPE [20]	sync. BFT	-	$O(N^2)$

Table 1: Comparison of randomness beacons. The *Model* column specifies what are the necessary assumptions for the protocol to work. The *Communication* columns specify communication complexities (per node) of a one-time setup phase (run at the very beginning), and of one query for fresh random bits. Some of them were designed for DKG and thus we do not specify the complexity of query (yet it can be made $O(N)$ in all cases). Some protocols also do not run any setup phase, in which case the complexity is omitted.

complexity. The approach of [29] uses the ideas of [14] and roughly follows the above outlined idea, but avoids the problem of consensus by working in a relaxed model: weak synchrony (which lies in between partial synchrony and full asynchrony).

Our Approach. We base our randomness beacon on threshold signatures, yet make a crucial observation that **full DKG is not necessary** for this purpose. Indeed what we run instead as setup (key generation phase) of our randomness beacon is a weaker variant of DKG. Very roughly: the end result of this variant is that the keys are dealt to a subset of at least $2f + 1$ nodes (thus possibly up to f nodes end up without keys). This allows us to construct a protocol with setup that incurs only $\tilde{O}(N^2)$ communication, whereas a variant with full DKG¹⁶ would require $\tilde{O}(N^3)$.

To obtain such a setup in an asynchronous setting we need to deal with the problem (that was sketched in the previous paragraph) of reaching consensus without having a common source of randomness in place. Intuitively a “disentanglement” of these two problems seems hard: in one direction this can be made formal via FLP Theorem [24] (consensus requires randomness); in the opposite direction, intuitively, generating a common coin toss requires all the nodes to agree on a particular random bit.

Canetti and Rabin present an interesting way to circumvent this problem in [19]: their consensus algorithm requires only a “weak coin”, i.e., a source of randomness that is common (among nodes) with constant, non-zero probability (and is allowed to give different results for different nodes otherwise). Then, via a beautiful construction they obtain such a randomness source that (crucially) does not require any consensus.

Our way to deal with this problem is different. Roughly speaking: in the setup each node “constructs” its unbiased source of randomness and reliably broadcasts it to the remaining nodes. Thus after

¹⁶We do not describe this variant in this paper; it can be obtained by replacing univariate by bivariate polynomials and adjusting the protocol accordingly (see [14]).

this step, the problem becomes to pick one out of N randomness sources (consensus). To this end we make a binary decision on each of these sources and pick as our randomness beacon the first that obtained a decision of “1”; however each of these binary consensus instances uses its own particular source of randomness (the one it is making a decision about). At the same time we make sure that the nodes that were late with showing their randomness sources to others will be always decided 0. While this is the basic idea behind our approach, there are several nontrivial gaps to be filled in order to obtain $\tilde{O}(N^2)$ communication complexity and $O(1)$ latency.

Other Designs. More recently, in the work of [46] a randomness source RandShare has been introduced, which runs a certain variant of Pedersen’s protocol to extract random bits. The protocol is claimed by the authors to work in the asynchronous model, yet fails to achieve that goal, as any protocol that **waits for** messages from **all the nodes** to proceed, fails to achieve liveness under asynchrony (or even under partial synchrony). In the Table 1 we list it as synchronous BFT, as after some adjustments it can be made to run in this model. In the same work [46] two other randomness beacons are also proposed: RandHound and RandHerd, yet both of them rely on strong, non-standard assumptions about the network and the adversary and thus we do not include them in Table 1. The method used by SCRAPE [20] at a high level resembles Pedersen’s protocol but requires access to a blockchain in order to have a total order on messages sent out during the protocol execution.

Finally, we mention an interesting line of work on generating randomness based on VDFs (Verifiable Delay Functions [7, 34, 42, 47]). Very roughly, the idea is to turn a biasable randomness (such as the hash of a Bitcoin block) into unbiased randomness via a VDF, i.e., a function $f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ that cannot be computed quickly and whose computation cannot be parallelized, yet it is possible to prove that $f(x) = y$ for a given y much faster than actually computing $f(x)$. The security of this approach relies on the assumption that the adversary cannot evaluate f at random inputs much faster than honest participants.

4.2 Randomness from Threshold Signatures

In this subsection we present the main cryptographic component of our construction: generating randomness using Threshold Signatures. When using a trusted dealer, this component is already enough to implement SecretBits, but the bulk of this section is devoted to proving that we can eliminate the need for a trusted dealer.

Randomness through Signatures. The main idea for generating randomness is as follows: suppose that there is a key pair (tk, vk) of private key and public key, such that tk is unknown, while vk is public, and tk allows to sign messages for some public-key cryptosystem that is deterministic¹⁷. (We refer to tk as to the “tossing key” while vk stands for “verification key”; we use these names to distinguish from the regular private-public key pairs held by the nodes.) Then, for any message m , its digital signature σ_m generated with respect to tk cannot be guessed, but can be verified using vk , thus $\text{hash}(\sigma_m) \in \{0, 1\}^\lambda$ provides us with λ random bits! There seems to be a contradiction here though: how can the tossing key

be secret and at the same time we are able to sign messages with it? Surprisingly, this is possible using *Threshold Cryptography*: the tossing key tk is “cut into pieces” and distributed among N nodes so that they can jointly sign messages using this key but no node (or a group of dishonest nodes) can learn tk .

More specifically, we use a threshold signature scheme built upon BLS signatures [8]. Such a scheme works over a GDH group G , i.e., a group in which the computational Diffie-Hellman problem is hard (i.e. computing g^{xy} given $g^x, g^y \in G$) but the decisional Diffie-Hellman problem is easy (i.e. verifying that $z = xy$ given $g^x, g^y, g^z \in G$). For more details and constructions of such groups we refer the reader to [8]. We assume from now on that G is a fixed, cyclic, GDH group generated by $g \in G$ and that the order of G is a large prime q . A tossing key tk in BLS is generated as a random element in $\mathbb{Z}_q$ and the public key is $y = g^{tk}$. A signature of a message m is simply $\sigma_m = \tilde{m}^{tk}$, where $\tilde{m} \in G$ is the hash (see [8] for a construction of such hash functions) of the message being a random element of G .

Distributing the Tossing Key. To distribute the secret tossing key among all nodes, the Shamir’s Secret Sharing Scheme [45] is employed. A trusted dealer generates a random tossing key tk along with a random polynomial A of degree f over $\mathbb{Z}_q$ such that $A(0) = tk$ and privately sends $tk_i = A(i)$ to every node $i = 1, 2, \dots, N$. The dealer also publishes verification keys, i.e., $VK = (g^{tk_1}, \dots, g^{tk_N})$. Now, whenever a signature of a message m needs to be generated, the nodes generate *shares* of the signature by signing m with their tossing keys, i.e., each node i multicasts $\tilde{m}^{A(i)}$. The main observation to make is that now $\tilde{m}^{A(0)}$ can be computed given only $\tilde{m}^{A(j)}$ for $f + 1$ different j ’s (by interpolation) and thus it is enough for a node P_j to multicast $\tilde{m}^{A(j)}$ as its share and collect f such shares from different nodes to recover σ_m . On the other hand, any collection of at most f shares is not enough to do that, therefore the adversary cannot sign m all by himself. For details, we refer to the pseudocode of ThresholdSignatures.

SecretBits Through Threshold Signatures. Given the just introduced machinery of threshold signatures, the SecretBits(i, r) primitive is straightforward to implement. Moreover, as promised in Section 3, we give here an implementation that is oblivious to its first argument, i.e., it does only depend on r , but not on i .

First of all, there is a setup phase whose purpose is to deal keys for generating secrets to all nodes. We start by giving a simple version in which an honest dealer is required for the setup. Subsequently, we explain how can this be replaced by a trustless setup, to yield the final version of SecretBits.

In the simpler case, the trusted dealer generates tossing keys and verification keys (TK, VK) for all the nodes using the GenerateKeys procedure, and then openly broadcasts VK to all nodes and to every node i he secretly sends the tossing key tk_i .

Given such a setup, when SecretBits(j, r) is executed by the protocol, every node i can simply ignore the j argument and generate

$$s_i(m) \leftarrow \text{CreateShare}(m, tk_i)$$

where m is a nonce determined from the round number, say $m = “r”$. Next, after creating its round- $(r + 1)$ unit U , the P_i collects all the

¹⁷ A system that for a given message m there exists only one correct signature for key pair (tk, vk) .

ThresholdSignatures:	
1	GenerateKeys():
2	Let $G = \langle g \rangle$ be a GDH group of prime order q
3	generate a random polynomial A of degree f over $\mathbb{Z}_q$
4	let $TK = (tk_1, \dots, tk_N)$ with $tk_i = A(i)$ for $i \in [N]$,
5	let $VK = (vk_1, \dots, vk_N)$ with $vk_i = g^{tk_i}$ for $i \in [N]$
6	output (TK, VK)
7	CreateShare(m, tk_i):
8	$\tilde{m} \leftarrow \text{hash}(m)$
9	output $\tilde{m}^{tk_i}$
10	VerifyShare(m, s, i, VK):
11	$\tilde{m} \leftarrow \text{hash}(m)$
12	/* can do the check below since G is GDH */
13	if $\log_g(vk_i) = \log_{\tilde{m}}(s)$ then
14	output True
15	else output False
16	GenerateSignature(m, S, VK):
17	/* Let $S = \{(s_i, i)\}_{i \in P}$ where $ P = f + 1$ */
18	/* Assume $\forall_j \text{VerifyShare}(m, s_j, i_j) = \text{True}$ */
19	interpolate $A(0)$, i.e., find $l_1, l_2, \dots, l_{f+1} \in \mathbb{Z}_q$ s.t.
20	$A(0) = \sum_{j=1}^{f+1} l_j A(i_j)$
21	$\sigma_m \leftarrow \prod_{j=1}^{f+1} s_j^{l_j}$
22	output σ_m

shares S included in U 's parents at round r and computes:

$$\sigma_m \leftarrow \text{GenerateSignature}(m, S, VK)$$

$$s(m) \leftarrow \text{hash}(\sigma_m)$$

and $s(m) \in \{0, 1\}^\lambda$ is meant as the output of $\text{SecretBits}(\cdot, r)$.

Finally, to get rid of the trusted dealer, in Subsection 4.3 we describe a trustless protocol that performs the setup instead of a trusted dealer. The crucial difference is that the keys are not generated by one entity, but jointly by all the nodes. Moreover, in the asynchronous version of the setup, every honest node i learns the verification key VK , some key tk_i and a set of "share dealers" $T \subseteq [N]$ of size $2f + 1$, such that every node P_i with $i \in T$ has a correct tossing key tk_i . This, while being slightly weaker than the outcome of the setup of trusted dealer, still allows to implement SecretBits as demonstrated in the below lemma.

LEMMA 4.1 (SECRET BITS). *The above scheme in both versions (with and without trusted dealer) correctly implements SecretBits .*

We refer the reader to Subsection E for a proof.

4.3 Randomness Beacon with Trustless Setup

In Section 4.2 we have discussed how to implement a Randomness Beacon based on a trusted dealer. Here, we devise a version of this protocol that has all the benefits of the previous one and at the same time is completely trustless. For the sake of clarity, we first provide a high level perspective of the new ideas and how do they combine to give a trustless Randomness Beacon, next we provide a short summary of the protocol in the form of a very informal

pseudocode, and finally we fill in the gaps by giving details on how the specific parts are implemented and glued together.

Key Boxes. Since no trusted dealer is available, perhaps the most natural idea is to let all the nodes serve as dealers simultaneously. More precisely we let every node emit (via RBC, i.e., by placing it as data in a unit) a tossing *Key Box* that is constructed by a node k (acting as a dealer) as follows:

- as in $\text{GenerateKeys}()$, sample a random polynomial

$$A_k(x) = \sum_{j=0}^f a_{k,j} x^j \in \mathbb{Z}_q[x]$$

of degree f ,

- compute a commitment to A_k as

$$C_k = (g^{a_{k,0}}, g^{a_{k,1}}, \dots, g^{a_{k,f}}).$$

- define the tossing keys TK_k and verification keys VK_k

$$tk_{k,i} := A_k(i) \quad \text{for } i = 1, 2, \dots, N$$

$$vk_{k,i} := g^{tk_{k,i}} \quad \text{for } i = 1, 2, \dots, N.$$

Note that in particular each verification key $vk_{k,i}$ for $i \in [N]$ can be computed from the commitment C_k as $vk_{k,i} = \prod_{j=0}^f C_{k,j}^{i^j}$.

- encrypt the tossing keys for every node i using the dedicated public key¹⁸ $pk_{k \rightarrow i}$ as

$$e_{k,i} := \text{Enc}_{k \rightarrow i}(tk_{k,i})$$

and let $E_k := (e_{k,1}, e_{k,2}, \dots, e_{k,N})$.

- the k th key box is defined as $KB_k = (C_k, E_k)$.

In our protocol, every node $\mathcal{P}_k$ generates its own key box KB_k and places (C_k, E_k) in his unit of round 0. We define the k th key set to be $KS_k = (VK_k, TK_k)$ and note that given the key box $KB_k = (C_k, E_k)$, every node can reconstruct VK_k and moreover, every node i can decrypt his tossing key $tk_{k,i}$ from the encrypted part E_k , but is not able to extract the remaining keys. The encrypted tossing keys E_k can be seen as a certain way of emulating "authenticated channels".

Verifying Key Sets. Since at least $2/3$ of nodes are honest, we also know that $2/3$ of all the key sets are safe to use, because they were produced by honest nodes who properly generated the key sets and the corresponding key boxes and erased the underlying polynomial (and thus all the tossing keys). Unfortunately, it is not possible to figure out which nodes cheated in this process (and kept the tossing keys that were supposed to be erased).

What is even harder to check, is whether a certain publicly known key box KB_k was generated according to the instructions above. Indeed, as a node $\mathcal{P}_i$ we have access only to our tossing key $tk_{k,i}$ and while we can verify that this key agrees with the verification key (check if $g^{tk_{k,i}} = vk_{k,i}$), we cannot do that for the remaining keys that are held by other nodes. The only way to perform verification of the key sets is to do that in collaboration with other nodes. Thus, in the protocol, there is a round at which

¹⁸We assume that as part of PKI setup each node i is given exactly N different key pairs for encryption: $(sk_{k \rightarrow i}, pk_{k \rightarrow i})$ for $k \in [N]$. The key $pk_{k \rightarrow i}$ is meant to be used by the k th node to encrypt a message whose recipient is i (denoted as $\text{Enc}_{k \rightarrow i}(\cdot)$). This setup is merely for simplicity of arguments – one could instead have one key per node if using verifiable encryption or double encryption with labels.

every node “votes” for correctness of all the key sets it has seen so far. As will be explained in detail later, these votes cannot be “faked” in the following sense: if a node $\mathcal{P}_i$ votes on a key set KS_k being incorrect, it needs to provide a proof that its key $tk_{k,i}$ (decrypted from $e_{k,i}$) is invalid, which cannot be done if $\mathcal{P}_k$ is an honest dealer. Thus consequently, dishonest nodes cannot deceive the others that a valid key set is incorrect.

Choosing Trusted Key Sets. At a later round these votes are collected and summarized locally by every node $\mathcal{P}_i$ and a trusted set of key sets $T_i \subseteq [N]$ is determined. Intuitively, T_i is the set of indices k of key sets such that:

- the node $\mathcal{P}_i$ is familiar with KB_k ,
- the node $\mathcal{P}_i$ has seen enough votes on KS_k correctness that it is certain that generating secrets from KS_k will be successful,

The second condition is determined based solely on the ch-DAG structure below the i th node unit at a particular round. What will be soon important is that, even though the sets T_i do not necessarily coincide for different i , it is guaranteed that $|T_i| \geq f + 1$ for every i , and thus crucially at least one honest key set is included in each of them.

Combining Tosses. We note that once the set T_i is determined for some index $i \in [N]$, this set essentially defines a global common source of randomness that cannot be biased by an adversary. Indeed, suppose we would like to extract the random bits corresponding to nonce m . First, in a given round, say r , every node should include its share for nonce m corresponding to every key set that it voted as being correct. In the next round, it is guaranteed that the shares included in round r are enough to recover the random secret $\sigma_{m,k} = m^{A_k(0)} \in G$ (the threshold signature of m generated using key set KS_k) for every $k \in T_i$. Since up to f out of these secrets might be generated by the adversary, we simply take

$$\tau_{m,i} := \prod_{k \in T_i} \sigma_{m,k} = m^{\sum_{k \in T_i} A_k(0)} \in G$$

to obtain a uniformly random element of G and thus (by hashing it) a random bitstring of length λ corresponding to node $\mathcal{P}_i$, resulting from nonce m . From now on we refer to the i th such source of randomness (i.e., corresponding to $\mathcal{P}_i$) as MultiCoin $_i$.

Agreeing on Common MultiCoin. So far we have said that every node $\mathcal{P}_i$ defines locally its strong source of randomness MultiCoin $_i$. Note however that paradoxically, this abundance of randomness sources is actually problematic: which one shall be used by all the nodes to have a “truly common” source of randomness? This is nothing other than a problem of “selecting a head”, i.e., choosing one unit from a round – a problem that we already solved in Section 3! At this point however, an attentive reader is likely to object, as the algorithm from Section 3 only works provided a common source of randomness. Therefore, the argument seems to be cyclic as we are trying to construct such a source of randomness from the algorithm from Section 3. Indeed, great care is required here: as explained in Section 3, all what is required for the ChooseHead protocol to work is a primitive SecretBits(i, r) that is supposed to inject a secret of λ bits at the r th round of the protocol. Not going too deep into details, we can appropriately implement such a method by employing the MultiCoins we have at our disposal. This part, along with the

construction of MultiCoins, constitutes the technical core of the whole protocol.

Combining Key Sets. Finally, once the head is chosen to be $l \in [N]$, from now on one could use MultiCoin $_l$ as the common source of randomness. If one does not care about savings in communication and computational complexity, then the construction is over. Otherwise, observe that tossing the MultiCoin $_l$ at a nonce m requires in the worst case N shares from every single node. This is sub-optimal, and here is a simple way to reduce it to just 1 share per node. Recall that every Key Set KS_k that contributes to MultiCoin $_l$ is generated from a polynomial $A_k \in \mathbb{Z}_q[x]$ of degree f . It is not hard to see that by simple algebraic manipulations one can combine the tossing keys and all the verification keys of all key sets KS_k for $k \in T_l$ so that the resulting Key Set corresponds to the sum of these polynomials

$$A(x) := \left(\sum_{k \in T_l} A_k(x) \right) \in \mathbb{Z}_q[x].$$

This gives a source of randomness that requires one share per nonce from every node; note that since the set T_l contains at least one honest node, the polynomial A can be considered random.

Protocol Sketch. We are now ready to provide an informal sketch of the Setup protocol and Toss protocol, see the ABFT – Beacon box below. The content of the box is mostly a succinct summary of Section 4.3. What might be unclear at this point is the condition of the if statement in the Toss function. It states that the tossing key tk_i is supposed to be correct in order to produce a share: indeed it can happen that one of the key boxes that is part of the MultiCoin $_l$ does not provide a correct tossing key for the i th node, in which case the combined tossing key tk_i cannot be correct either. This however is not a problem, as the protocol still guarantees that at least $2f + 1$ nodes hold correct combined tossing keys, and thus there will be always enough shares at our disposal to run ExtractBits.

4.4 Details of the Setup

We provide some more details regarding several components of the Setup phase of ABFT – Beacon that were treated informally in the previous subsection.

Voting on Key Sets. Just before creating the unit at round 3 the i th node is supposed to inspect all the key boxes that are present in its current copy of the ch-DAG. Suppose KB_k for some $k \in [N]$ is one of such key sets. The only piece of information about KB_k that is known to $\mathcal{P}_i$ but hidden from the remaining nodes is its tossing key. Node $\mathcal{P}_i$ recovers this key by decrypting it using its secret key (dedicated for k) $sk_{k \rightarrow i}$

$$tk_{k,i} \leftarrow \text{Dec}_{k \rightarrow i}(e_{k,i}).$$

Now, if node $\mathcal{P}_k$ is dishonest, it might have included an incorrect tossing key, to check correctness, the i th node verifies whether

$$g^{tk_{k,i}} \stackrel{?}{=} vk_{k,i},$$

where g is the fixed generator of the group G we are working over. The i th node includes the following piece of information in its round-3 unit

$$\text{VerKey}(KB_k, i) = \begin{cases} 1 & \text{if } tk_{k,i} \text{ correct} \\ \text{Dec}_{k \rightarrow i}(e_{k,i}) & \text{oth.} \end{cases}$$

ABFT-Beacon:	
1	Setup():
	/* Instructions for node $\mathcal{P}_i$. */
2	Initialize growing the ch-DAG $\mathcal{D}$
3	In the data field of your units include (as specified in Section 4.4):
4	At round 0: a key box KB_i ,
5	At round 3: votes regarding correctness of key boxes present in $\mathcal{D}$,
6	At rounds ≥ 6: shares necessary to extract randomness from SecretBits.
7	Run ChooseHead to determine the head unit at round 6 and let l be its creator.
8	Combine the key sets $\{KS_j : j \in T_l\}$ and let (tk_i, VK) be the corresponding tossing key and verification keys.
9	Toss(m):
	/* Code for node $\mathcal{P}_i$. */
10	if tk_i is correct then
11	$s_i \leftarrow \text{CreateShare}(m, tk_i)$
12	multicast (s_i, i)
13	wait until receiving a set of $f + 1$ valid shares S
	/* validity is checked using VerifyShare() */
14	output $\sigma_m := \text{ExtractBits}(m, S, VK)$

Note that if a node $\mathcal{P}_i$ votes that KB_k is incorrect (by including the bottom option of VerKey in its unit) it cannot lie, since other nodes can verify whether the plaintext it provided agrees with the ciphertext in KB_k (as the encryption scheme is assumed to be deterministic) and if that is not the case, they treat a unit with such a vote as invalid (and thus not include it in their ch-DAG). Thus, consequently, the only way dishonest nodes could cheat here is by providing positive votes for incorrect key boxes. This can not harm honest nodes, because by positively verifying some key set a node declares that from now on it will be providing shares for this particular key set whenever requested. If in reality the corresponding tossing key is not available to this node, it will not be able to create such shares and hence all its units will be henceforth considered invalid.

One important property this scheme has is that it is safe for an honest recipient $\mathcal{P}_i$ of $e_{k,i}$ to reveal the plaintext decryption of $e_{k,i}$ in case it is not (as expected) the tossing key $tk_{k,i}$ – indeed if $\mathcal{P}_k$ is dishonest then either he actually encrypted some data d in $e_{k,i}$ in which case he learns nothing new (because d is revealed) or otherwise he obtained $e_{k,i}$ by some other means, in which case $\text{Dec}_{k \rightarrow i}(e_{k,i})$ is a random string, because no honest node ever creates a ciphertext encrypted with $p_{k \rightarrow i}$ and we assume that the encryption scheme is semantically secure. Note also that if instead of having N key pairs per node we used a similar scheme with every node having just one key pair, then the adversary could reveal some tossing keys of honest nodes through the following attack: the adversary copies an honest node's (say j th) ciphertext $e_{j,i}$ and includes it as $e_{k,i}$ in which case $\mathcal{P}_i$ is forced to reveal $\text{Dec}_i(e_{j,i}) = tk_{j,i}$ which should have remained secret! This is the reason why we need dedicated key pairs for every pair of nodes.

Forming MultiCoins. The unit $V := U[i; 6]$ created by the i th node in round 6 defines a set $T_i \subseteq [N]$ as follows: $k \in N$ is considered an element of T_i if and only if all the three conditions below are met

- (1) $U[k; 0] \leq V$,
- (2) For every $j \in [N]$ such that $U[j; 3] \leq V$ it holds that $\text{VerKey}(KB_k, j) = 1$.

At this point it is important to note that every node that has $U[i; 6]$ in its copy of the ch-DAG can compute T_i as all the conditions above can be checked deterministically given only the ch-DAG.

SecretBits via MultiCoins. Recall that to implement CommonVote and GeneratePermutation it suffices to implement a more general primitive SecretBits(i, r) whose purpose is to inject a secret at round r that can be recovered by every node in round $r + 1$ but cannot be recovered by the adversary till at least one honest node has created a unit of round r .

The technical subtlety that becomes crucial here is that the SecretBits(i, r) is only called for $r \geq 9$ and only by nodes that have $U[i; 6]$ in their local ch-DAG (see Lemma E.3). More specifically, this allows us to implement SecretBits($i, \cdot$) through MultiCoin $_i$. The rationale behind doing so is that every node that sees $U[i; 6]$ can also see all the Key Sets that comprise the i th MultiCoin and thus consequently it “knows” what MultiCoin $_i$ is¹⁹.

Suppose now that we would like to inject a secret at round r for index $i \in [N]$. Define a nonce $m := \text{"i||r"}$ and request all the nodes $\mathcal{P}_k$ such that $U[i; 6] \leq U[k, r]$ to include in $U[k, r]$ a share for the nonce m for every Key Set KS^j such that $j \in T_i$. In addition, if $\mathcal{P}_k$ voted that KS_j is incorrect in round 3, or $\mathcal{P}_k$ did not vote for KB_j at all (since KB_j was not yet available to him at round 3) then $\mathcal{P}_k$ is not obligated to include a share (note that its unit $U[k, 3]$ that is below $U[k, r]$ contains evidence that its key was incorrect).

As we then show in Section 4.4, given any unit $U \in \mathcal{D}$ of round $r + 1$ one can then extract the value of MultiCoin $_i$ from the shares present in round- r units in $\mathcal{D}$. Thus, consequently, the value of SecretBits(i, r) in a ch-DAG $\mathcal{D}$ is available whenever any unit in $\mathcal{D}$ has round $\geq r + 1$, hence we arrive at

LEMMA 4.2 (SECRET BITS FROM MULTICOINS). *The above defined scheme based on MultiCoins correctly implements SecretBits.*

The proof of the above lemma is provided in Section E.

5 ACKNOWLEDGEMENTS

First of all, authors would like to thank Matthew Niernerg for introducing us to the topic, constant support, and countless hours spent on valuable discussions. Additionally, we would like to show our gratitude to Michał Handzlik, Tomasz Kisielewski, Maciej Gawron, and Łukasz Lachowski, for reading the paper, proposing changes that improved its consistency and readability, and discussions that helped us to make the paper easier to understand.

This research was funded by the Aleph Zero Foundation.

¹⁹We emphasize that using a fixed MultiCoin, for instance MultiCoin $_1$ would not be correct here, as there is no guarantee the 1st node has delivered its unit at round 6. More generally, it is crucial that for different units U_0 we allow to use different MultiCoins, otherwise we would have solved Byzantine Consensus without randomness, which is impossible by the FLP Theorem [24].

REFERENCES

- [1] Michael Abd-El-Malek, Gregory R. Ganger, Garth R. Goodson, Michael K. Reiter, and Jay J. Wylie. 2005. Fault-scalable Byzantine fault-tolerant services. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles 2005, SOSP 2005, Brighton, UK, October 23-26, 2005*. 59–74. <https://doi.org/10.1145/1095810.1095817>
- [2] Ittai Abraham, Guy Gueta, Dahlia Malkhi, Lorenzo Alvisi, Ramakrishna Kotla, and Jean-Philippe Martin. 2017. Revisiting Fast Practical Byzantine Fault Tolerance. CoRR abs/1712.01367 (2017). arXiv:1712.01367 <http://arxiv.org/abs/1712.01367>
- [3] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. 2019. Asymptotically Optimal Validated Asynchronous Byzantine Agreement. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*. 337–346. <https://doi.org/10.1145/3293611.3331612>
- [4] Leemon Baird. 2016. The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. *Swirlds Tech Reports SWIRLDS-TR-2016-01, Tech. Rep.* (2016).
- [5] Michael Ben-Or and Ran El-Yaniv. 2003. Resilient-optimal interactive consistency in constant time. *Distributed Computing* 16, 4 (2003), 249–262. <https://doi.org/10.1007/s00446-002-0083-3>
- [6] Alexandra Boldyreva. 2003. Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme. In *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*. 31–46. https://doi.org/10.1007/3-540-36288-6_3
- [7] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. 2018. Verifiable Delay Functions. In *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part I*. 757–788. https://doi.org/10.1007/978-3-319-96884-1_25
- [8] Dan Boneh, Ben Lynn, and Hovav Shacham. 2004. Short Signatures from the Weil Pairing. *J. Cryptology* 17, 4 (2004), 297–319. <https://doi.org/10.1007/s00145-004-0314-9>
- [9] Gabriel Bracha. 1987. Asynchronous Byzantine Agreement Protocols. *Inf. Comput.* 75, 2 (1987), 130–143. [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X)
- [10] Gabriel Bracha and Sam Toueg. 1983. Resilient Consensus Protocols. In *Proceedings of the Second Annual ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, Montreal, Quebec, Canada, August 17-19, 1983*. 12–26. <https://doi.org/10.1145/800221.806706>
- [11] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2018. The latest gossip on BFT consensus. CoRR abs/1807.04938 (2018). arXiv:1807.04938 <http://arxiv.org/abs/1807.04938>
- [12] Vitalik Buterin and Virgil Griffith. 2017. Casper the Friendly Finality Gadget. CoRR abs/1710.09437 (2017). arXiv:1710.09437 <http://arxiv.org/abs/1710.09437>
- [13] Christian Cachin, Rachid Guerraoui, and Luis E. T. Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming* (2. ed.). Springer. <https://doi.org/10.1007/978-3-642-15260-3>
- [14] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strohli. 2002. Asynchronous verifiable secret sharing and proactive cryptosystems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS 2002, Washington, DC, USA, November 18-22, 2002*. 88–97. <https://doi.org/10.1145/586110.586124>
- [15] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and Efficient Asynchronous Broadcast Protocols. In *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*. 524–541. https://doi.org/10.1007/3-540-44647-8_31
- [16] Christian Cachin, Klaus Kursawe, and Victor Shoup. 2005. Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement Using Cryptography. *J. Cryptology* 18, 3 (2005), 219–246. <https://doi.org/10.1007/s00145-005-0318-0>
- [17] Christian Cachin and Stefano Tessaro. 2005. Asynchronous verifiable information dispersal. In *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*. IEEE, 191–201.
- [18] Christian Cachin and Marko Vukolic. 2017. Blockchain Consensus Protocols in the Wild. CoRR abs/1707.01873 (2017). arXiv:1707.01873 <http://arxiv.org/abs/1707.01873>
- [19] Ran Canetti and Tal Rabin. 1993. Fast asynchronous Byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing, May 16-18, 1993, San Diego, CA, USA*. 42–51. <https://doi.org/10.1145/167088.167105>
- [20] Ignacio Cascudo and Bernardo David. 2017. SCRAPE: Scalable Randomness Attested by Public Entities. In *Applied Cryptography and Network Security - 15th International Conference, ACNS 2017, Kanazawa, Japan, July 10-12, 2017, Proceedings*. 537–556. https://doi.org/10.1007/978-3-319-61204-1_27
- [21] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*. 173–186. <https://dl.acm.org/citation.cfm?id=296824>
- [22] Miguel Correia, Nuno Ferreira Neves, and Paulo Verissimo. 2006. From Consensus to Atomic Broadcast: Time-Free Byzantine-Resistant Protocols without Signatures. *Comput. J.* 49, 1 (2006), 82–96. <https://doi.org/10.1093/comjnl/bxh145>
- [23] George Danezis and David Hrycyszyn. 2018. Blockmania: from Block DAGs to Consensus. *arXiv preprint arXiv:1809.01620* (2018).
- [24] Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (1985), 374–382. <https://doi.org/10.1145/3149.214121>
- [25] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. 2003. Secure Applications of Pedersen's Distributed Key Generation Protocol. In *Topics in Cryptology - CT-RSA 2003, The Cryptographers' Track at the RSA Conference 2003, San Francisco, CA, USA, April 13-17, 2003, Proceedings*. 373–390. https://doi.org/10.1007/3-540-36563-X_26
- [26] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. 2007. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *J. Cryptology* 20, 1 (2007), 51–83. <https://doi.org/10.1007/s00145-006-0347-3>
- [27] Vassos Hadzilacos and Sam Toueg. 1994. *A modular approach to fault-tolerant broadcasts and related problems*. Technical Report. Cornell University.
- [28] Timo Hanke, Mahnush Movahedi, and Dominic Williams. 2018. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548* (2018).
- [29] Aniket Kate, Yizhou Huang, and Ian Goldberg. 2012. Distributed Key Generation in the Wild. *IACR Cryptology ePrint Archive* 2012 (2012), 377. <http://eprint.iacr.org/2012/377>
- [30] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. 2017. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I*. 357–388. https://doi.org/10.1007/978-3-319-63688-7_12
- [31] Ramakrishna Kotla, Lorenzo Alvisi, Michael Dahlin, Allen Clement, and Edmund L. Wong. 2009. Zyzzyva: Speculative Byzantine fault tolerance. *ACM Trans. Comput. Syst.* 27, 4 (2009), 7:1–7:39. <https://doi.org/10.1145/1658357.1658358>
- [32] Jae Kwon and Ethan Buchman. [n.d.]. A Network of Distributed Ledgers. ([n.d.]). <https://cosmos.network/cosmos-whitepaper.pdf>
- [33] Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM* 21, 7 (1978), 558–565. <https://doi.org/10.1145/359545.359563>
- [34] Arjen K. Lenstra and Benjamin Wesolowski. 2017. Trustworthy public randomness with sloth, unicorn, and trx. *IJACT* 3, 4 (2017), 330–343. <https://doi.org/10.1504/IJACT.2017.10010315>
- [35] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The Honey Badger of BFT Protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. 31–42. <https://doi.org/10.1145/2976749.2978399>
- [36] Zarko Milosevic, Martin Hutle, and André Schiper. 2011. On the Reduction of Atomic Broadcast to Consensus with Byzantine Faults. In *30th IEEE Symposium on Reliable Distributed Systems (SRDS 2011), Madrid, Spain, October 4-7, 2011*. 235–244. <https://doi.org/10.1109/SRDS.2011.36>
- [37] Louise E. Moser and P. M. Melliar-Smith. 1999. Byzantine-Resistant Total Ordering Algorithms. *Inf. Comput.* 150, 1 (1999), 75–111. <https://doi.org/10.1006/inco.1998.2770>
- [38] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. 2015. Signature-Free Asynchronous Binary Byzantine Consensus with $t < n/3$, $O(n^2)$ Messages, and $O(1)$ Expected Time. *J. ACM* 62, 4 (2015), 31:1–31:21. <https://doi.org/10.1145/2785953>
- [39] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).
- [40] Rafael Pass and Elaine Shi. 2017. Hybrid Consensus: Efficient Consensus in the Permissionless Model. In *31st International Symposium on Distributed Computing, DISC 2017, October 16-20, 2017, Vienna, Austria*. 39:1–39:16. <https://doi.org/10.4230/LIPIcs.DISC.2017.39>
- [41] Torben P. Pedersen. 1991. A Threshold Cryptosystem without a Trusted Party (Extended Abstract). In *Advances in Cryptology - EUROCRYPT '91, Workshop on the Theory and Application of Cryptographic Techniques, Brighton, UK, April 8-11, 1991, Proceedings*. 522–526. https://doi.org/10.1007/3-540-46416-6_47
- [42] Krzysztof Pietrzak. 2019. Simple Verifiable Delay Functions. In *10th Innovations in Theoretical Computer Science Conference, ITCS 2019, January 10-12, 2019, San Diego, California, USA*. 60:1–60:15. <https://doi.org/10.4230/LIPIcs.ITCS.2019.60>
- [43] David Pointcheval and Jacques Stern. 1996. Security Proofs for Signature Schemes. In *Advances in Cryptology - EUROCRYPT '96, International Conference on the Theory and Application of Cryptographic Techniques, Saragossa, Spain, May 12-16, 1996, Proceedings*. 387–398. https://doi.org/10.1007/3-540-68339-9_33
- [44] Serguei Popov. 2016. The tangle. *cit. on* (2016), 131.
- [45] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. <https://doi.org/10.1145/359168.359176>
- [46] Ewa Syta, Philipp Jovanovic, Eleftherios Kokoris-Kogias, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Michael J. Fischer, and Bryan Ford. 2017. Scalable Bias-Resistant Distributed Randomness. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. 444–460. <https://doi.org/10.1109/SP.2017.444>

- [47] Benjamin Wesolowski. 2019. Efficient Verifiable Delay Functions. In *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III*. 379–407. https://doi.org/10.1007/978-3-030-17659-4_13

A PRACTICAL CONSIDERATIONS

The protocol that we described in the main body of the paper has excellent theoretical properties and achieves optimal asymptotic guarantees, however in the original form might not be viable for practical implementation. The high level reason for that is that it was designed to operate in the harshest possible adversarial setting (i.e. the adversary controlling f out of $3f + 1$ nodes and being able to arbitrarily delay messages) and it was not optimized for the "optimistic case". This means intuitively that as of now, the protocol can withstand arbitrarily oppressive "attacks" of the adversary, but does not get any faster when no attacks take place.

In this section we present several adjustments to the protocol that allow us to make it significantly faster in the optimistic case, without compromising its strong security guarantees. As a consequence, we obtain a version of the protocol that is well suited for practical implementation – we show in particular (see Section C.3) that under partial synchrony it matches the optimistic 3-round validation delay of PBFT and Tendermint [11, 21].

Below, we list the main sources of practical inefficiency of the protocol:

- (1) The reliance on RBC: each DAG-round effectively takes 3 async-rounds, so in practice the protocol requires more rounds than simple synchronous or partially synchronous protocols such as PBFT. Furthermore, performing $\Omega(N)$ instances of RBC simultaneously by a node, forces it to send $\Omega(N^2)$ distinct messages per round, which might not be feasible.
- (2) The worst-case assumption that the adversary can pretty much arbitrarily manipulate the structure of the ch-DAG forces the randomness to be revealed with large delay and causes inefficiency.
- (3) The total size of metadata (total size of units, ignoring transactions) produced in one round by all the nodes is quite large: $\Omega(N^2\lambda)$ bits, because each round- r unit contains hashes of $\Omega(N)$ units from round $r - 1$.

We show how to solve these three issues in Sections A.1, A.2 and A.3 respectively.

A.1 From RBC to Multicast and Random Gossip

As a practical solution, we propose to use a combination of multicast and random gossip in place of RBC to share the ch-DAG between nodes. More specifically, the following Quick-DAG-Grow algorithm is meant to replace the DAG-Grow from the main body (based on RBC).

At this point we emphasize that while we believe that our proposal yields a truly efficient algorithm, it is rather cumbersome to formally reason about the communication complexity of such a solution (specifically about the gossip part) and we do not attempt

Quick-DAG-Grow($\mathcal{D}$):

```

1 CreateUnit(data):
2   for  $r = 0, 1, 2, \dots$  do
3     if  $r > 0$  then
4       wait until  $|\{U \in \mathcal{D} : R(U) = r - 1\}| \geq 2f + 1$ 
5        $P \leftarrow \{\text{maximal } \mathcal{P}_i\text{'s unit of round } < r \text{ in } \mathcal{D} : \mathcal{P}_i \in \mathcal{P}\}$ 
6       create a new unit  $U$  with  $P$  as parents
7       include data in  $U$ 
8       add  $U$  to  $\mathcal{D}$ 
9       multicast  $U$ 
10 ReceiveUnits:
11   loop forever
12     upon receiving a unit  $U$  do
13       if  $U$  is correct then add  $U$  to buffer  $\mathcal{B}$ 
14       while exists  $V \in \mathcal{B}$  whose all parents are in  $\mathcal{D}$  do
15         move  $V$  from  $\mathcal{B}$  to  $\mathcal{D}$ 
16 Gossip:
17   /* Run by node  $i$  */
18   loop forever
19      $j \leftarrow$  randomly sampled node
20     send  $j$  concise info about  $\mathcal{D}_i$ 
21     receive all units in  $\mathcal{D}_j \setminus \mathcal{D}_i$ 
22     send all units in  $\mathcal{D}_i \setminus \mathcal{D}_j$ 

```

to provide a formal treatment. Instead, in a practical implementation one needs to include a set of rules that make it impossible for malicious nodes to request the same data over and over again (in order to slow down honest nodes). To analyze further effects of switching from RBC to multicast + gossip, recall that RBC in the original protocol allows us to solve the following two problems

- (1) the **data availability** problem, as a given piece of data is locally output by an honest node in RBC only when it is guaranteed to be eventually output by every other honest node,
- (2) the problem of **forks**, since only one version of each unit may be locally output by an honest node.

The data availability problem, we discuss in Section A.1.1. Regarding forks: their existence in the ch-DAG does not pose a threat to the theoretical properties of our consensus protocol. Indeed, in Section C.2 we show that the protocol is guaranteed to reach consensus even in the presence of forks in ch-DAG. The only (yet serious) problem with forks is that if malicious nodes produce a lot of them and all these forks need to be processed by honest nodes, this can cause crashes due to lack of resources, i.e., RAM or disc space. To counter this issue we add an auxiliary mechanism to the protocol whose purpose is bounding the number of possible forks (see Section A.1.2). Additionally, we show that without a very specific set of precautions, such an attack is rather simple to conduct and, to best of our knowledge, affects most of the currently proposed DAG-based protocols. We formalize it as a *Fork Bomb* attack in section H.

A.1.1 Ensuring data availability via gossip. Suppose for a moment that we have multicast as the only mechanism for sending

units to other nodes (i.e., the creator multicasts his newly created unit to all the nodes). While this is clearly the fastest way to disseminate information through a network, it is certainly prone to adversarial attacks on data availability. This is a consequence of the fact that only the creator of this unit would then share it with other nodes. Hence, if the creator is malicious, it may introduce intentional inconsistencies in local copies of ch-DAGs stored by honest nodes by sending newly created units only to some part of the network or even sending different variants to different nodes. To see that such an attack can not be trivially countered, let us explore possible scenarios, depending on the particular rules of unit creation:

- If node i is **allowed** to choose a unit U as a parent for its newly created unit V despite the fact that it does not know the whole “context” of U (i.e., it does not have all of U ’s parents in its local copy of ch-DAG), then we risk that the functions $\text{Vote}(V, \cdot)$ and $\text{UnitDecide}(V, \cdot)$ will never succeed to terminate with a non- $\perp$ verdict. Hence, node i may never be able to make a decision about V .
- If on the other hand i is **not allowed** to choose U as a parent in such a case, then the growth of ch-DAG may stall. Indeed, another honest node might have created U with a parent W that was available to him but not i (because W was created by a malicious node).

Hence it seems that to counter such intentional or unintentional (resulting for example from temporary network failures) inconsistencies, there has to be a mechanism in place that allows nodes to exchange information about units they did not produce. To this end in the Quick-DAG-Grow protocol each node regularly reconciliates its local version of the ch-DAG with randomly chosen peers, in a gossip-like fashion. Since each two honest nodes sync with each other infinitely often (with probability 1), this guarantees data availability.

A.1.2 Bounding the number of forks. We introduce a mechanism that bounds the number of possible forks to N variants (or rather “branches”) per node.

Note that if a (dishonest) node starts broadcasting a forked unit, this is quickly noticed by all honest nodes, and the forker can be banned after such an incident. Note however that we cannot reliably “rollback” the ch-DAG to a version before the fork has happened, as two different honest nodes might have already build their units on two different fork branches. Still, since units are signed by their creators, the forker can be proven to be malicious and punished accordingly, for example by slashing its stake. This means that the situation when a malicious node just creates a small number of variants of a unit is not really dangerous – it will be detected quite soon and this node will be banned forever.

What might be potentially dangerous though is when a group of malicious nodes collaborate to build a multi-level “Fork Bomb” (Section H) composed of a huge number of units and the honest nodes are forced to download them all, which could lead to crashes. This is the main motivation behind introducing the *alert protocol*.

We note that sending a unit via RBC can be thought of as “committing” to a single variant of a unit. In the absence of RBC the simplest attack of an adversary would be to send a different variant of a unit to every node. Since just after receiving this unit, honest

nodes cannot be aware of all the different variants, each of them might legally use a different variant as its parent. This means that there is no way to bound the number of different variants of one forked unit in the ch-DAG by less than N and we would like to propose a mechanism that does not allow to create more (while without any additional rules there might be an exponential number of variants – see Section H).

In the solution we propose every node must “commit” to at most one variant of every received unit, by analogy to what happens in RBC. The general rule is that honest nodes can accept only those units that someone committed to. By default (when no forks are yet detected), the commitment will be simply realized by creating a next round unit that has one particular variant as its parent. On the other hand, if a fork is observed, then every node will have to send its commitment using RBC, to prevent attacks akin to the fork bomb (Section H).

A node can work in one of two states: normal and alert. In the normal state, the node multicasts its own units, gossips with other nodes, and builds its local copy of the ch-DAG. Now assume that a node k detects a fork, i.e., obtains two distinct variants U_1 and U_2 of the round- r unit created by i . Then node k enters the alert mode. It stops building the ch-DAG and broadcasts a special message $\text{Alert}(k, i)$ using RBC. The message consists of three parts:

- (1) Proof of node i being malicious, i.e., the pair (U_1, U_2) .
- (2) The hash and the round number of the unit of highest round created by node i that is currently present in $\mathcal{D}_k$ (possibly null).
- (3) The alert id (every node assigns numeric ids to its alerts, i.e., $id = 0, 1, 2, \dots$ and never starts a new alert before finishing the previous one).

By broadcasting the hash, node k commits to a single variant of a whole chain of units created by node i , up to the one of highest round, currently known by node k . Since up to this point node k did not know that node i was malicious, then indeed units created by node i in $\mathcal{D}_k$ form a chain.

Implementing this part requires some additional care, since now units cannot be validated and added one by one. Also, the sender should start with proving that the highest unit actually exists (by showing the unit and the set of its parents), so a malicious node cannot send arbitrarily large chunks of units, that cannot be parsed individually.

At this point we also note that since every node performs its alerts sequentially (by assigning numbers to them), in the worst case there can be at most $O(N)$ alerts run in parallel.

We now provide details on how a node should behave after a fork is discovered. Assume that node k is honest, and i is malicious (forking).

- (1) Node k stops communicating with node i immediately after obtaining a proof that i is malicious.
- (2) Immediately after obtaining a proof that i is malicious, node k enters the alert mode, and broadcasts $\text{Alert}(k, i)$.
- (3) Node k can issue at most one $\text{Alert}(k, \cdot)$ at once, and alerts must be numbered consecutively.
- (4) Node k cannot participate in $\text{Alert}(j, \cdot)$ if it has not locally terminated all previous alerts issued by node j .

- (5) Node k exits alert mode immediately after $\text{Alert}(k, i)$ ends locally for k , and stores locally the proof of finishing the alert, along with the proof that node i is malicious.
- (6) After exiting the alert mode, node k can still accept units created by i , but only if they are below at least one unit that some other node committed to.

The Quick-DAG-Grow protocol enriched by the alert mechanism implemented through the above rules can serve as a reliable way to grow a ch-DAG that provably contains at most N forks per node. More specifically, in Section B.2 we prove the following

THEOREM A.1. *Protocol Quick-DAG-Grow with alert system is reliable and growing (as in definition 3.2). Additionally, every unit can be forked at most N times.*

A.2 Adjustments to the Consensus Mechanism

In Aleph protocol the mechanism of choosing round heads is designed in a way such that no matter whether the adversary is involved or not, the head is chosen in an expected constant number of rounds. While this is optimal from the theoretical perspective, in practice we care a lot about constants, and for this reason we describe a slight change to the consensus mechanism that allows us to achieve latency of 3 rounds under favorable network conditions (which we expect to be the default) and still have worst-case constant latency.

The key changes are to make the permutation (along which the head is chosen) “a little” deterministic and to change the pattern of initial deterministic common votes.

Permutation. Recall that in order to choose the head for round r in Aleph first a random permutation over units at this round π_r is commonly generated and then in the order determined by π_r the first unit decided 1 is picked as the head. The issue is that in order for the randomness to actually “trick” the adversary, and not allow him to make the latency high, the permutation can be revealed at earliest at round $r + 4$ in case of Aleph, and $r + 5$ in case of QuickAleph²⁰. In particular the decision cannot be made earlier than a round after the permutation is revealed. To improve upon that in the optimistic case, we make the permutation π_r partially deterministic and allow to recover the first entry of the permutation already in round r . More specifically, the mechanism of determining π_r is as follows:

- Generate pseudo-randomly an index $i_0 \in [N]$ based on just the round number r or on the head of round $r - 1$.
- Let τ_r be a random permutation constructed as previously using $\text{SecretBits}(\cdot, r + 5)$.
- In π_r as first come all units created by node i_0 in round r (there might be multiple of them because of forking), sorted by hashes, and then come all the remaining units of round r in the order as determined by τ_r .

Such a design allows the nodes to learn i_0 and hence the first candidate for the head right away in round r and thus (in case of positive decision) choose it already in round $r + 3$. The remainder of the permutation is still random and cannot be manipulated by the adversary, thus the theoretical guarantees on latency are preserved.

²⁰The difference stems from the potential existence of forks in ch-DAG constructed in QuickAleph and will become more clear after reading lemma C.11

For completeness we provide pseudocode for the new version of $\text{GeneratePermutation}$. In the pseudocode DefaultIndex can be any deterministic function that is computed from $\mathcal{D}$ after the decision on round $r - 1$ has been made. Perhaps the simplest option is $\text{DefaultIndex}(r, \mathcal{D}) := 1 + (r \bmod N)$. In practice we might want to use some more complex strategy, which promotes nodes creating units that spread fast; for instance, it could depend on the head chosen for the $(r - 1)$ -round.

GeneratePermutation($r, \mathcal{D}$):

```

1 if  $R(\mathcal{D}) < r + 3$  then output  $\perp$ 
2  $i_0 \leftarrow \text{DefaultIndex}(r, \mathcal{D})$ 
3  $(V_1, V_2, \dots, V_l) \leftarrow$  list of units created by  $\mathcal{P}_{i_0}$  in round  $r$  sorted by hashes
   /* Typically  $l = 1$ ,  $l > 1$  can only happen if  $\mathcal{P}_{i_0}$  forked in round  $r$ . */
4 for each unit  $U$  of round  $r$  in  $\mathcal{D}$  do
5    $i \leftarrow$  the creator of  $U$ 
6    $x \leftarrow \text{SecretBits}(i, r + 5, \mathcal{D})$ 
7   if  $x = \perp$  then output  $(V_1, V_2, \dots, V_l)$ 
8   assign  $\text{priority}(U) \leftarrow \text{hash}(x || U) \in \{0, 1\}^\lambda$ 
9 let  $(U_1, U_2, \dots, U_k)$  be the units in  $\mathcal{D}$  of round  $r$  not created by  $\mathcal{P}_{i_0}$  sorted by  $\text{priority}(\cdot)$ 
10 output  $(V_1, V_2, \dots, V_l, U_1, U_2, \dots, U_k)$ 

```

Common Votes. In QuickAleph we use a slightly modified sequence of initial deterministic common votes as compared to Aleph. This change is rather technical but, roughly speaking, is geared towards making the head decision as quickly as possible in the protocol. Here is a short summary of the new CommonVote scheme. Let U_0 be a unit created by i and $r := R(U_0)$, then for $d = 2, 3, \dots$ we define

$$\text{CommonVote}(U_0, r + d) := \begin{cases} 1 & \text{for } d = 2 \\ 0 & \text{for } d = 3 \\ \text{SecretBits}(i, r + d + 1) & \text{for } d \geq 4 \end{cases}$$

Where in the last case we extract a single random bit from SecretBits , as previously.

A.3 Reducing size of units

We note that encoding parents of a unit by a set of their hashes is rather inefficient as it takes roughly $N \cdot \lambda$ bits to store just this information (λ bits per parent). Since the reasonable choices for λ in this setting are 256 or 512, this is a rather significant overhead. In this section we propose a solution that reduces this to just a small constant number (i.e. around 2) of bits per parent.

Recall that in the absence of forks every unit is uniquely characterized by its creator id and by round number. Since forking is a protocol violation that is severely penalized (for instance, by slashing the node’s stake), one should expect it to happen rarely if at all. For this reason it makes sense to just use this simple encoding of units: (i, r) (meaning the unit created by i at round r) and add a simple fallback mechanism that detects forked parents. More specifically, the parents of a unit U are encoded as a combination of the following two pieces of data

- (1) A list $L_U = [r_1, r_2, \dots, r_N]$ of length N that contains the round numbers of parents of U corresponding to creators $1, 2, \dots, N$. In the typical situation the list has only a small number of distinct elements, hence can be compressed to just 2 or 3 bits per entry.
- (2) The “control hash”

$$h_U := h(h_1 || h_2 || \dots || h_N),$$

where $h_1, h_2, \dots, h_N$ are the hashes of parents of U .

Consequently, the above encoding requires $O(N + \lambda)$ bits instead of $\Omega(N\lambda)$ bits as the original one. While such a construction does not allow to identify forks straight away, it does allow to identify inconsistencies. Indeed, if an honest node k receives a unit U from node j , then it will read the parent rounds and check if all parent units are already present in $\mathcal{D}_k$. If not, then these units will be requested from the sender of U , and failing to obtain them will result in dropping the unit U by node k . In case k knows all parent units, it may verify the control hash and if inconsistency is detected, node k should request²¹ all parent units present in $\mathcal{D}_j$ from node j . If node j is also honest, then node k will reveal a forking unit among parents of U and issue an alert before adding U to $\mathcal{D}_k$. On the other hand, in case $\mathcal{P}_j$ does not provide parents of U , $\mathcal{P}_k$ can simply drop the unit U .

B ANALYSIS OF PROTOCOLS CONSTRUCTING CH-DAGS

Within the paper, two possible protocols constructing ch-DAG are considered, namely, DAG-Grow and Quick-DAG-Grow. In this section, we analyze these protocols in the context of desirable properties as in definition 3.2.

B.1 DAG-Grow

We provide a proof of Theorem 3.1, i.e., we show that the DAG-Grow protocol is reliable, ever-expanding, fork-free and advances the DAG rounds at least as quickly as the asynchronous are progressing.

PROOF OF THEOREM 3.1.

Reliable. Node $\mathcal{P}_i$ adds a unit U to its ch-DAG only if it is locally output by ch-RBC. By the properties of ch-RBC we conclude that if a unit is locally output for one honest node, it is eventually locally output by every other honest node.

Ever-expanding. Assume the contrary, i.e., there exists r such that no honest node $\mathcal{P}_i$ can gather $2f + 1$ units of round r in $\mathcal{D}_i$ and hence no node is able to produce a unit of round $r + 1$. Let r_0 be the minimum such r . We know that $r_0 > 0$, since every honest node participates in broadcasting of units of round 0 without waiting for parents. The units of round $r_0 - 1$ created by honest nodes are eventually added to local copies of all honest nodes, hence at some point, every honest node can create and broadcast a unit of round r , which then is eventually included in all local copies of ch-DAG. As there are $2f + 1$ honest nodes, we arrive at a contradiction.

Fork-free. This is a direct consequence of the “reliability” of ch-RBC, see Lemma F.1 (Fast Agreement)

²¹The request should include signed hashes of all units from $\mathcal{D}_k$ that are indeed parents of U to prevent k from sending false requests.

DAG rounds vs asynchronous rounds. By Lemma D.1 each honest node produces at least one new unit each 5 async-rounds. Since each new unit needs to have higher DAG-round, it concludes the proof. $\square$

B.2 Quick-DAG-Grow

Since Quick-DAG-Grow does not rely on reliable broadcast, it does not enjoy all the good properties of the DAG-Grow protocol. Most notably, it allows nodes to process forks and add forked units to the ch-DAG. We prove that it is still reliable and ever-growing, and additionally that the number of forks created by a single node is bounded by N for each round, i.e., we prove Theorem A.1.

PROOF OF THEOREM A.1.

Reliable. First, we observe that while the alert system may slow the protocol down for some time, it may not cause it to stall indefinitely. Indeed, each honest node needs to engage in at most N alert RBC instances per each discovered forker, i.e., at most $\frac{1}{3}N^2$ instances in total. Since an RBC instance started by an honest node is guaranteed to terminate locally for each node, there is some point in time of the protocol execution, name it T_0 , after which no more alerts instantiated by honest nodes are active anymore. Consequently, honest nodes can gossip their local copies of ch-DAG after T_0 without any obstacles. This means in particular that every pair of honest nodes exchange their ch-DAG information infinitely often (with probability one), which implies reliability.

Ever-expanding. This follows from reliability as the set of honest is large enough ($2f + 1$) to advance the rounds of the ch-DAG by themselves. Indeed, by induction, for every round number $r > 0$ every honest node eventually receives at least $2f + 1$ units created by non-forkers (for instance honest nodes) in round $r - 1$ and hence can create a correct unit of round r by choosing these units as parents.

Bounding number of forks. An honest node i adds a fork variant to its local copy of ch-DAG only in one of the two scenarios:

- It was the first version of that fork that i has received,
- Some other node has publicly committed to this version via the alert system.

Since there are only $N - 1$ other nodes that could cause alerts, the limit of maximum of N versions of a fork follows. $\square$

C ANALYSIS OF CONSENSUS PROTOCOLS

The organization of this section is as follows: In Subsection C.1 we analyze the consensus mechanism in the Aleph protocol. In particular we show that ChooseHead is consistent between nodes and incurs only a constant number of rounds of delay in expectation. This also implies that every unit (created by an honest node) waits only a constant number of asynchronous rounds until it is added to the total order, which is necessary in the proof of Theorem 2.1. The Subsection C.2 is analogous but discusses QuickAleph (introduced in Section A) instead. Finally in Subsection C.3 we show an optimistic bound of 3 rounds validation for QuickAleph.

Since the proofs in Subsection C.2 are typically close adaptations of these in Subsection C.1 we recommend the reader to start with the latter. In the analysis of QuickAleph we distinguish between two cases: when the adversary attacks using forks, and when no

forks have happened recently. In the former case the latency might increase from expected $O(1)$ to $O(\log N)$ rounds, yet each time this happens at least one of the nodes gets banned²² and thus it can happen only f times during the whole execution of the protocol. We do not attempt to optimize constants in this section, but focus only on obtaining optimal asymptotic guarantees. A result of practical importance is obtained in Subsection C.3 where we show that 3 rounds are enough in the "optimistic" case.

Throughout this section we assume that the SecretBits primitive satisfies the properties stated in Definition 3.3. An implementation of SecretBits is provided in Section 4.4.

C.1 Aleph

We would to prove that the expected number of rounds needed for a unit to be ordered by the primitive OrderUnits is constant, and that each honest node is bound to produce the same ordering. To achieve this, first we need a series of technical lemmas.

LEMMA C.1 (VOTE LATENCY). *Let X be a random variable which, for a unit U_0 , indicates the number of rounds after which all units vote unanimously on U_0 . Formally, let U_0 be a unit of round r and define $X = X(U_0)$ to be the smallest l such that there exists $\sigma \in \{0, 1\}$ such that for every unit U of round $r + l$ we have $\text{Vote}(U_0, U) = \sigma$. Then for $K \in \mathbb{N}$ we have*

$$P(X \geq K) \leq 2^{4-K}.$$

PROOF. Fix a unit U_0 . Let $r' > r + 4$ be a round number and let $\mathcal{P}_k$ be the first honest node to create a unit U of the round r' . Let σ denote a vote on U_0 that was cast by at least $f + 1$ units in $\downarrow(U)$. Then, every unit of round r' will have at least one parent that votes σ . It is easy to check that if also $\sigma = \text{CommonVote}(U_0, r', \mathcal{D}_k)$, then every unit of round r' is bound to vote σ .

By Definition 3.3 (1), the votes of units in $\downarrow(U)$ are independent of $\text{CommonVote}(U_0, r', \mathcal{D}_k)$ since it uses $\text{SecretBits}(i, r', \mathcal{D}_k)$. Therefore, if at round $r' - 1$ voting was not unanimous, then with probability at least $1/2$ it will be unanimous starting from round r' onward. Since $P(X \geq 5) \leq \frac{1}{2}$, we may calculate $P(X \geq K) \leq 2^{-K+4}$, for $K > 4$ by induction, and observe that it is trivially true for $K = 1, 2, 3, 4$. $\square$

LEMMA C.2 (DECISION LATENCY). *Let Y be a random variable that, for a unit U_0 , indicates the number of rounds after which all honest nodes decide on U_0 . Formally, let U_0 be a unit of a round r and define $Y = Y(U_0)$ to be the smallest l such that there exists σ such that for every honest node $\mathcal{P}_i$ if $R(\mathcal{D}_i) \geq r + l$, then $\text{Decide}(U_0; \mathcal{D}_i) = \sigma$. Then for $K \in \mathbb{N}, K > 0$ we have*

$$P(Y \geq K) \leq \frac{K}{2^{K-5}} = O(K \cdot 2^{-K}).$$

PROOF. First, we need to check that Y is well-defined, i.e. for every U_0 there is $l \in \mathbb{N}$ and there is σ such that for every honest node $\mathcal{P}_i$ if $R(\mathcal{D}_i) \geq r + l$, then $\text{Decide}(U_0; \mathcal{D}_i) = \sigma$.

We observe that if there is an honest node $\mathcal{P}_k$ and a unit $U \in \mathcal{D}_k$ such that $\text{UnitDecide}(U_0, U, \mathcal{D}_k) = \sigma \neq \perp$, then eventually for every honest node $\mathcal{P}_i$, we will have $\text{Decide}(U_0; \mathcal{D}_i) = \sigma$. Indeed, fix an honest node $\mathcal{P}_k$ and a unit $U \in \mathcal{D}_k$ of round $r' > r + 4$

such that $\text{UnitDecide}(U_0, U, \mathcal{D}_k) = \sigma \neq \perp$. Then, at least $2f + 1$ units of round $r' - 1$ vote σ and $\text{CommonVote}(U_0, r') = \sigma$. Now by the definitions of UnitVote, we see that every other unit U' of round r' must have $\text{UnitVote}(U_0, U', \mathcal{D}_k) = \sigma$, and thus by a simple induction we get that, every unit of round greater than r' will also vote σ on U_0 . Finally, by the definition of Decide we see that once UnitDecide outputs a $\sigma \neq \perp$, then the result of UnitVote is σ and does not change with the growth of the local copy of ch-DAG.

By the above, if we prove that UnitDecide will always eventually output non- $\perp$, then the definition of Y will be correct.

Let $Y' = Y'(U_0)$ be defined as the smallest l such that for every unit U of round $r + l$ the function $\text{UnitDecide}(U_0, U)$ outputs σ . If $X(U_0) = l$, then either $Y'(U_0) = l$, or $Y'(U_0)$ is the index of the first round after l at which CommonVote equals σ . The probability that at round $r + l$ nodes vote unanimously σ for the first time, but then no CommonVote was equal to σ before round $r + L$ equals $P(X = l)/2^{L-1-l}$, and thus we have:

$$\begin{aligned} P(Y' \geq L) &\leq \sum_{l=0}^{L-1} 2^{-L+1+l} P(X = l) + P(X \geq L) \\ &= \sum_{l=0}^{L-1} 2^{-L+1+l} P(X = l) + \sum_{l=L}^{+\infty} P(X = l) \\ &= 2^{-L} P(X \geq 0) + \sum_{l=0}^{L-1} 2^{-L+l} P(X \geq l) \\ &\leq \frac{L+1}{2^{L-4}} \end{aligned}$$

Since X is well-defined, then so is Y' and we see that UnitDecide eventually outputs a non- $\perp$. Moreover, observe that $Y \leq Y' + 1$. Indeed, if every node of round $r + l$ has decided σ , then the ch-DAG of height $\geq r + l + 1$ includes at least one of the decisions, and we can read the last required secret. Finally, $Y \leq Y' + 1$ implies

$$P(Y \geq K) \leq P(Y' \geq K - 1) \leq \frac{K}{2^{K-5}}.$$

$\square$

The following lemma guarantees that at every level there are a lot of units that are decided on 1.

LEMMA C.3 (FAST POSITIVE DECISIONS). *Assume that an honest node $\mathcal{P}_i$ has just created a unit U of round $r + 3$. At this point in the protocol execution, there exists a set S_r of at least $2f + 1$ units of round r such that for every $U_0 \in S_r$, every honest node will eventually have $\text{Decide}(U_0, \mathcal{D}_i) = 1$.*

PROOF. Let K be a set of $2f + 1$ nodes that created units in $\downarrow(U)$. Additionally, let $\mathcal{T}$ be the set of $2f + 1$ units of round $r + 1$ created by nodes in K . Every unit in $\downarrow(U)$ is above $2f + 1$ units of round $r + 1$, hence it is above at least $f + 1$ units in $\mathcal{T}$ (the remaining f units may be created by nodes outside of K). By the pigeonhole principle, there exists a unit $U_0 \in \mathcal{T}$ such that at least $f + 1$ units in $\downarrow(U)$ are above it. Set $S_r := \downarrow(U_0)$.

Let V be a unit of round $r + 3$ and $V' \in \downarrow(V)$ be its parent that is above U_0 (which has to exist since every subset of $2f + 1$ units of round $r + 2$ must include at least one unit above U_0). Since $\text{CommonVote}(U_r, W)$ equals 1 for all W of rounds $r + 1, r + 2, r + 3$,

²²Banning forks is implemented by adding one simple rule to the growing protocol: never pick forking nodes as parents.

for $U_r \in S_r$ we have $\text{Vote}(U_r, U_0) = 1$, hence $\text{Vote}(U_r, V') = 1$, and finally, $\text{Vote}(U_r, V) = 1$.

Thus, during all subsequent rounds $\text{Vote}(U_r, \cdot) = 1$ and U_r will be decided 1 as soon as the next `CommonVote` equals 1. $\square$

Intuitively, this lemma states that the set of potential heads that will be eventually positively decided is large. Additionally, it is defined relatively quickly, i.e., before the adversary sees the content of any unit of round $r + 3$. Importantly, it is defined before the permutation in which potential heads will be considered is revealed. Note that it has some resemblance to the spread protocol defined in [5].

In general, the above result cannot be improved, as the adversary can always slow down f nodes, thus their units, unseen by others, may not be considered “ready” to put in the linear order. Also, note that this lemma does not require any common randomness, as the votes in round $r + 3$ depend only on the structure of the ch-DAG.

LEMMA C.4 (FAST NEGATIVE DECISIONS). *Let U be a unit of round r such that for some honest node i , $U \notin \mathcal{D}_i$ even though $R(\mathcal{D}_i) \geq r + 4$. Then, for any local view $\mathcal{D}$, $\text{Decide}(U, \mathcal{D}) \neq 1$.*

PROOF. Assume for contradiction that $\mathcal{D}$ is local view such that $\text{Decide}(U, \mathcal{D}) = 1$. By definition of `Decide`, there has to be unit $V_1 \in \mathcal{D}$ such that $\text{UnitDecide}(U, V_1, \mathcal{D}) = 1$. Let now $\mathcal{D}'$ be a local view such that $\mathcal{D}, \mathcal{D}_i \subseteq \mathcal{D}'$. Such $\mathcal{D}'$ is possible to construct since all local views have to be compatible by point (2) of Theorem 3.1.

Let V_0 be a unit of DAG-round $R(\mathcal{D}_i) + 4$ in $\mathcal{D}_i$ (and hence in $\mathcal{D}'$). Since V_0 can't be above U , for every $V \in \downarrow(V_0)$ we have $\text{Vote}(U, V, \mathcal{D}_i) = 0$. Since $|\downarrow(V_0)| \geq 2f + 1$, each unit of DAG-round $r + 4$ in $\mathcal{D}'$ is above at least one unit in $\downarrow(V_0)$ and hence votes either 0 or by `CommonVote`. But `CommonVote` for U always equals 0 at DAG-round $R(U) + 4$ by definition, and consequently, each unit of DAG-round $r + 4$ has to Vote 0 for U . If all units votes unanimously at any given DAG-round, such vote is passed over to the next DAG-rounds contradicting the existence of V_1 deciding 1, what concludes the proof. $\square$

FACT C.1. *Let $X_1, \dots, X_M$ be random variables, $K \in \mathbb{R}$. Then*

$$P(\max(X_1, \dots, X_M) \geq K) \leq \sum_{m=1}^M P(X_m \geq K)$$

PROOF. Simply observe that if $\max(X_1, \dots, X_M) \geq K$, then for at least one $m \in [M]$ it must be $X_m \geq K$

$$\{\max(X_1, \dots, X_M) \geq K\} \subseteq \bigcup_{m=1}^M \{X_m \geq K\}.$$

$\square$

The next lemma shows the bound on the number of additional rounds that are needed to choose a head.

LEMMA C.5 (CHOOSEHEAD LATENCY). *The function `ChooseHead` satisfies the following properties:*

- **Agreement.** *For every round r there is a uniquely chosen head U , i.e., for every ch-DAG $\mathcal{D}$ maintained by an honest node, $\text{ChooseHead}(r, \mathcal{D}) \in \{\perp, U\}$.*

- **Low latency.** *Let Z_r be a random variable defined as the smallest l such that for every local copy $\mathcal{D}$ of height $r + l$ we have $\text{ChooseHead}(r, \mathcal{D}) \neq \perp$. Then for $K \in \mathbb{N}, K > 0$ we have $P(Z_r \geq K) = O\left(K \cdot 2^{-K}\right)$*

PROOF.

Agreement. Suppose for the sake of contradiction that there exist two ch-DAGs $\mathcal{D}_i$ and $\mathcal{D}_j$ maintained by honest nodes i, j , such that $\text{ChooseHead}(r, \mathcal{D}_i) = U$ and $\text{ChooseHead}(r, \mathcal{D}_j) = U'$ for two distinct units U, U' . Note that for this we necessarily have

$$R(\mathcal{D}_i) \geq r + 4 \quad \text{and} \quad R(\mathcal{D}_j) \geq r + 4,$$

as otherwise `GeneratePermutation`($r, \cdot$) would return $\perp$.

Further, as necessarily $\text{Decide}(U, \mathcal{D}_i) = 1$ and $\text{Decide}(U', \mathcal{D}_j) = 1$, by Lemma C.4 we obtain that both U and U' need to be present in $\mathcal{D}_i$ and $\mathcal{D}_j$.

To conclude the argument it remains to observe that the priorities computed by `GeneratePermutation`($r, \cdot$) are consistent between different ch-DAGs as they are computed deterministically based on the common outcome of `SecretBits`. We thus arrive at the desired contradiction: if, say, U has higher priority than U' then the j th node should have chosen U instead of U' .

Constant latency. Consider the set S_r from Lemma C.3. Let $\mathcal{P}_i$ be a node, $x = \text{SecretBits}(i, r + 4, \mathcal{D}_i)$, and let

$$\pi_r = (U_1, \dots, U_k) = \text{GeneratePermutation}(r, \mathcal{D}_i).$$

The permutation given by priorities $\text{hash}(x || U_i)$ for $i = 1, 2, \dots, N$ is uniformly random by Definition 3.3 (1) and the assumption that hash gives random bits in $\{0, 1\}^\lambda$. Moreover, it is independent of the set S_r , as this set is defined when the first unit of round $r + 3$ is created and π_r was unknown before it.

Let S_r denote a random variable defined as the smallest index s such that s -th element of the permutation π_r is in S_r . Since the permutation is uniformly random and independent of S_r , then for $s = 1, \dots, f + 1$ we have

$$P(S_r = s) = \frac{2f + 1}{3f + 2 - s} \prod_{j=1}^{s-1} \frac{f + 1 - j}{3f + 2 - j},$$

and $P(S_r > f + 1) = 0$, hence for $s = 1, \dots, k$, we have

$$P(S_r = s) \leq 3^{-s+1}.$$

By Lemma C.3, all units from S_r must be decided 1, then to calculate `ChooseHead`($r, \mathcal{D}_i$) we need to wait for decisions on at most S_r units of round r . Using Fact C.1 for random variables from Lemma C.2 we see that for $K > 4$

$$P(Z_r \geq K \mid S_r = s) \leq \sum_{s=1}^k P(Y(U_s) \geq K) \leq s \cdot \frac{K}{2^{K-5}},$$

therefore

$$P(Z_r \geq K) \leq \sum_{s=1}^N P(S_r = s) P(Z_r \geq K \mid S_r = s) = O\left(K \cdot 2^{-K}\right).$$

$\square$

We end this section with a theorem that shows how long does it take to append a head to the linear order.

THEOREM C.6 (ORDERUNITS LATENCY). *Let W_r be a random variable that indicates number of rounds required to append a head of round r to the linear order. Formally, W_r is defined as the smallest l such that for every local copy $\mathcal{D}_i$ of height $r + l$ we have*

$$\text{ChooseHead}(r, \mathcal{D}_i) \in \text{OrderUnits}(\mathcal{D}_i).$$

Then

$$\mathbb{E}(W_r) = O(1)$$

PROOF. The first part of the proof will show that for all $K \in \mathbb{N}, K > 0$ we have

$$P(W_r \geq K) = O\left(K \cdot 2^{-K}\right).$$

Let $\mathcal{P}_i$ be a node. $\text{ChooseHead}(r, \mathcal{D}_i) \in \text{OrderUnits}(\mathcal{D}_i)$ implies that for all $j = 0, 1, \dots, r$ we have

$$\text{ChooseHead}(j, \mathcal{D}_i) \neq \perp.$$

Therefore, if $W_r \geq K$, then for at least one round j we have $Z_j \geq K + r - j$, where Z_j is a random variable from Lemma C.5. Since

$$(W_r \geq K) \subseteq \bigcup_{j=0}^r (Z_j \geq K + r - j),$$

then

$$\begin{aligned} P(W_r \geq K) &\leq \sum_{j=0}^r P(Z_j \geq K + r - j) \leq O(1) \sum_{j=0}^r \frac{K + j}{2^{K+j}} \\ &= O(1) \sum_{j=K}^{K+r} \frac{j}{2^j} \leq O(1) \sum_{j=K}^{+\infty} \frac{j}{2^j} = O\left(K \cdot 2^{-K}\right). \end{aligned}$$

Since W_r has values in $\mathbb{N}$, then

$$\mathbb{E}(W_r) = \sum_{K=1}^{+\infty} P(W_r \geq K),$$

and finally we have

$$\mathbb{E}(Z_r) = \sum_{K=1}^{+\infty} P(Z_r \geq K) = O(1) \sum_{K=1}^{+\infty} \frac{K}{2^K} = O(1).$$

□

C.2 QuickAleph

In this section we analyze the QuickAleph consensus mechanism. The main difference when compared to Aleph is that now we allow the adversary to create forks. Below, we introduce two definitions that allow us to capture forking situations in the ch-DAG so that we can prove how the latency of the protocol behaves in their presence. At the end of this subsection we also show that these situations are rare and can happen only $O(N)$ times.

DEFINITION C.1. *Let r be a round number. We say that a unit U of round $r + 4$ is a fork witness if it is above at least two variants of the same unit of round r or $r + 1$.*

DEFINITION C.2. *Let $\mathcal{U}$ be a set of all $2f + 1$ units of round $r + 4$ created by honest nodes. We say that the r th round of the ch-DAG is publicly forked, if there are at least $f + 1$ fork witnesses among units $\mathcal{U}$.*

We proceed with the analysis of the latency of QuickAleph starting again by determining the delay in rounds until all nodes start to vote unanimously.

LEMMA C.7 (VOTE LATENCY [FOR QuickAleph]). *Let X be a random variable that for a unit U_0 represents the number of rounds after which all units unanimously vote on U_0 . Formally, let U_0 be a unit of round r and define $X = X(U_0)$ to be the smallest l such that for some $\sigma \in \{0, 1\}$, for every unit U of round $r + l$ we have $\text{Vote}(U_0, U) = \sigma$. Then for $K \in \mathbb{N}$ we have*

$$P(X \geq K) \leq \left(\frac{3}{4}\right)^{(K-6)/2}.$$

PROOF. Fix a unit U_0 . Let $r' > r + 5$ be an odd round number and let $\mathcal{P}_k$ be the first honest node to create a unit U of round r' .

Let σ denote a vote such that there is no unit $V \in \downarrow(U)$ for which every unit in $\downarrow(V)$ votes σ on U_0 . The value of σ is well-defined, i.e., at least one value from $\{0, 1\}$ satisfies this property (pick $\sigma = 0$ if both 0 and 1 satisfy the above property). Indeed, every two units $V, V' \in \downarrow(U)$ have at least one parent unit in common, thus it is not possible that every unit in $\downarrow(V)$ votes σ , and every unit in $\downarrow(V')$ votes $1 - \sigma$.

Recall that the adversary cannot predict $\text{CommonVote}(U_0, r' - 1)$ or $\text{CommonVote}(U_0, r')$ before U is created. Now, if

$$\text{CommonVote}(U_0, r' - 1, \mathcal{D}_k) = \text{CommonVote}(U_0, r', \mathcal{D}_k) = 1 - \sigma,$$

then every unit in $\downarrow(U)$ votes $1 - \sigma$ on U_0 , and so does U . If some other unit U' of round r' were to vote σ on U_0 , then it would mean that at least $2f + 1$ units in $\downarrow(U')$ vote σ on U_0 , which is necessary to overcome the r' -round common vote. But this is impossible, as this set of $2f + 1$ units must include at least one unit from $\downarrow(U)$. Therefore, if at round $r' - 2$ voting was not unanimous, then with probability at least $1/4$ it will be unanimous starting from round r' onward. Hence, we may check $P(X \geq K) \leq \left(\frac{3}{4}\right)^{(K-6)/2}$, for $K > 5$ by induction, and observe that it is trivially true for $K = 1, 2, 3, 4, 5$. □

Here we obtain a slightly worse bound than in the corresponding Lemma C.1 for the Aleph protocol. The main reason for that is that the property that units U and U' have at least $f + 1$ common parent units does not longer hold, as now up to f of them may be forked. One can try to improve this result by introducing different voting rules, and analyzing forked and non-forked cases separately, but this would only make the argument cumbersome, and would not help to decrease the asymptotic latency in the asynchronous case, nor the numerical latency in the partially synchronous case.

LEMMA C.8 (DECISION LATENCY [FOR QuickAleph]). *Let Y be a random variable that for a unit U_0 indicates the number of rounds after which all honest nodes decide on U_0 . Formally, let U_0 be a unit of a round r and define $Y = Y(U_0)$ to be the smallest l such that there exists σ such that for every honest node $\mathcal{P}_i$ if $\text{R}(\mathcal{D}_i) \geq r + l$, then $\text{Decide}(U_0; \mathcal{D}_i) = \sigma$. Then for $K \in \mathbb{N}$ we have*

$$P(Y \geq K) = O\left(K \cdot \left(\frac{3}{4}\right)^{K/2}\right).$$

PROOF. The proof is analogous to that of Lemma C.2. First, we show that Y is well-defined. The argument is exactly the same, not affected by introducing forks. If at least one unit decides a non- $\perp$ value σ , then every unit of equal or higher round votes σ , and eventually there will be a round with common vote equal to σ , which will trigger the decision among all nodes.

We define $Y' = Y'(U_0)$ as in the proof of Lemma C.2 and obtain

$$\begin{aligned} P(Y' \geq L) &\leq 2^{-L} P(X \geq 0) + \sum_{l=0}^{L-1} 2^{-L+l} P(X \geq l) \\ &\leq (L+1) \left(\frac{3}{4}\right)^{(L-6)/2} \end{aligned}$$

The final difference is that now $Y \leq Y' + 2$, because we need one additional round to read the value of the last required secret:

$$P(Y \geq K) \leq P(Y' \geq K-2) \leq (K-1) \left(\frac{3}{4}\right)^{(K-8)/2}.$$

□

Similarly as for Aleph, for $O(1)$ latency it is crucial to show that a large set of units of round r is guaranteed to be decided positively.

LEMMA C.9 (FAST POSITIVE DECISIONS [FOR QuickAleph]). *Let $r \geq 0$ be a round number. Assume that an honest node $\mathcal{P}_i$ has created a unit U of round $r+4$. There exists at least one unit U_0 of round r such that $\text{Decide}(U_0, \mathcal{D}_i) = 1$. If additionally U is not a fork witness, then there exists a set $\mathcal{S}_{i,r}$ of at least $1.5f+1$ units of round r such that for every $U_0 \in \mathcal{S}_{i,r}$ we have $\text{Decide}(U_0, \mathcal{D}_i) = 1$.*

PROOF. For convenience, denote by $\mathcal{U}_{r'}$ the set of all units of round r' below U .

Let us first observe that whenever for a unit U_0 every unit $V \in \mathcal{U}_{r+2}$ votes 1 on U_0 , then it is decided 1 by U , i.e.,

$$(\forall V \in \mathcal{U}_{r+2} \text{ Vote}(U_0, V) = 1) \Rightarrow \text{UnitDecide}(U_0, U) = 1,$$

This can be seen as follows:

- every unit $W \in \mathcal{U}_{r+3}$ votes 1, i.e., $\text{Vote}(U_0, W) = 1$ because $\downarrow(W)$ unanimously vote 1,
- since the common vote at round $r+4$ is 1 and all round- $(r+3)$ parents of U unanimously vote 1, $\text{UnitDecide}(U_0, U) = 1$.

The remaining part of the proof is to show that in the general case there is at least one unit $U_0 \in \mathcal{U}_r$ that is voted 1 by every unit in $\mathcal{U}_{r+2}$ and that there is at least $1.5f+1$ such units in the case when U is not a forking witness.

For the first part, note that every unit W of round $r+1$ has at least $f+1$ honest parents at round r , thus by reverse counting there exists a unit U_0 of round r , created by an honest node that is a parent of at least $f+1$ honest units at round $r+1$. Note that U_0 is as desired, i.e., every unit V of round $r+2$ votes 1 for U_0 because at least one honest unit $W \in \downarrow(V)$ is above U_0 .

For the second part, suppose that U is not a fork witness, in which case the sets $\mathcal{U}_r$ and $\mathcal{U}_{r+1}$ contain no forks and hence if we

denote

$$\begin{aligned} a &:= N - |\mathcal{U}_r|, \\ b &:= N - |\mathcal{U}_{r+1}|, \end{aligned}$$

then $0 \leq a, b \leq f$. We define the set $\mathcal{S}_{i,r}$ to be all units $U_0 \in \mathcal{U}_r$ that are parents of at least $f-b+1$ units from $\mathcal{U}_{r+1}$.

Suppose now that U_0 is such a unit, we show that all units in $\mathcal{U}_{r+2}$ vote 1 on U_0 . Indeed, take any $V \in \mathcal{U}_{r+2}$. Since V has at least $2f+1$ parents in $\mathcal{U}_{r+1}$, at least one of them is above U_0 (by definition of $\mathcal{S}_{i,r}$) and thus $\downarrow(V)$ does not unanimously vote on 0. The default vote at round $r+2$ is 1 therefore V has to vote 1 on U_0 , as desired.

Finally, it remains to show that $\mathcal{S}_{i,r}$ is large. For brevity denote the number of elements in $\mathcal{S}_{i,r}$ by x . We start by observing that the number of edges connecting units from $\mathcal{U}_r$ and $\mathcal{U}_{r+1}$ is greater or equal than

$$(2f+1) \cdot |\mathcal{U}_{r+1}| = (2f+1) \cdot (N-b).$$

On the other hand, we can obtain an upper bound on the number of such edges by assuming that each unit from $\mathcal{S}_{i,r}$ has the maximum number of $N-b$ incoming edges and each unit from $\mathcal{U}_r \setminus \mathcal{S}_{i,r}$ has the maximum number of $f-b$ incoming edges, thus:

$$x \cdot (N-b) + (N-a-x)(f-b) \geq (2f+1)(N-b),$$

which after simple rearrangements leads to

$$x \geq 2f+1 - \frac{(f-a)(f-b)}{2f+1}.$$

Since $0 \leq a, b \leq f$ we obtain

$$x \geq 1.5f+1.$$

□

It is instructive to compare Lemma C.9 regarding QuickAleph to Lemma C.3 regarding Aleph. Note that in the non-forked case every unit from the set $\mathcal{U}_{r+2}$ has at least $2f+1$ parents in the set $\mathcal{U}_{r+1}$, and by pigeonhole principle must be above all units from $\mathcal{S}_{i,r}$. Therefore, we decrease the number of rounds after which units from $\mathcal{S}_{i,r}$ become “popular” from 3 to 2. This is achieved at the cost of reducing the size of this set by $0.5f$ units, but is necessary to speed up the protocol in the optimistic case.

LEMMA C.10 (FAST NEGATIVE DECISIONS [FOR QuickAleph]). *Let $r \geq 0$ be a round number. Let U be a unit of round $r+3$, and $U_0 \not\leq U$ be a unit of round r . Then, for any local view $\mathcal{D} \ni U_0$ with $R(\mathcal{D}) \geq r+5$ we have $\text{Decide}(U_0, \mathcal{D}) = 0$.*

PROOF. Let U' be any unit of round $r+3$. Then U' votes 0 on U_0 . Indeed, if U' were to vote 1, then there would be at least $2f+1$ units in $\downarrow(U')$ above U_0 , and therefore at least one unit in $\downarrow(U)$ above U_0 , but this would imply $U \geq U_0$. Hence, every unit of round greater or equal to $r+3$ votes 0 on U_0 , and finally every unit of round $r+5$ decides 0 on U_0 due to the common vote being equal to 0. □

In QuickAleph the latency of choosing head can differ depending on whether we are in the forking situation or not. The following lemma states this formally.

LEMMA C.11 (CHOOSEHEAD LATENCY [FOR QuickAleph]). *The function ChooseHead satisfies the following properties:*

- **Agreement.** For every round r there is a uniquely chosen head U , i.e., for every ch-DAG $\mathcal{D}$ maintained by an honest node, $\text{ChooseHead}(r, \mathcal{D}) \in \{\perp, U\}$.
- **Low latency.** Let Z_r be a random variable defined as the smallest l such that for every local copy $\mathcal{D}$ of height $r + l$ we have $\text{ChooseHead}(r, \mathcal{D}) \neq \perp$. Then for $K \in \mathbb{N}$ we have

$$P(Z_r \geq K) = O\left(K \cdot \left(\frac{3}{4}\right)^{K/2}\right)$$

if the protocol is locally non-forked at round r , and

$$P(Z_r \geq K) = O\left(N^2 \cdot K \cdot \left(\frac{3}{4}\right)^{K/2}\right)$$

otherwise.

PROOF. **Agreement.** Similarly as in the corresponding part of the proof of Lemma C.5 we observe that the permutation (computed by different nodes) along which the head at a given round is chosen is consistent in the sense that: if two nodes both hold a pair of units U_0, U_1 of round r then these units will end up obtaining the same priorities at both nodes and thus will be placed in the same relative order in the permutation.

Consequently all we need to show is that at the time when an honest node i decides a unit U as the head of round r then every unit U' that is not part of its local copy $\mathcal{D}_i$ will be decided 0. This indeed follows from Lemma C.10 since for $\text{ChooseHead}(r, \mathcal{D}_i)$ to not be $\perp$ the ch-DAG $\mathcal{D}_i$ needs to have $R(\mathcal{D}_i) \geq r + 3$.

Low latency. The proof of the non-forked case differs from the proof of Lemma C.5 in two ways. Firstly, the set of units that will eventually be decided 1 is smaller, but still has size $\Omega(N)$. Secondly, the first unit in the permutation is known in advance. Here we prove a slightly stronger result, namely we assume that the first $c \geq 0$ units are known, where c should be thought of as a small constant (independent of N). For concreteness one can simply think of $c = 1$.

Let $r \geq 0$ be a round number. Let U be the first $(r + 5)$ -round unit created by an honest node. If the protocol is locally non-forked at round r , then there are at least $f + 1$ $(r + 4)$ -round units that are not fork witnesses, hence there is at least one unit $V \in \downarrow(U)$, created by node i , that is not a fork witness.

Let $\mathcal{U}_r$ denote all round- r units below V , and consider the set $\mathcal{S}_{i,r} \subseteq \mathcal{U}_r$ from Lemma C.9. The set $\mathcal{U}_r$ has at most N elements and contains no forks. From Lemma C.10 we know that all round- r units not in $\mathcal{U}_r$ (possibly including some forks of units from $\mathcal{U}_r$) are decided 0 by every node at round $r + 5$. Therefore, we can only consider units from $\mathcal{U}_r$ when calculating $\text{GeneratePermutation}(r)$. Note that the unit priorities are independent from sets $\mathcal{S}_{i,r}, \mathcal{U}_r$, because the unit V exists before the adversary can read $\text{SecretBits}(\cdot, r + 5)$ using shares from the unit U .

Let $s_{i,r}$ denote a random variable defined as the smallest index s such that s -th element of the permutation on units $\mathcal{U}_r$ is in $\mathcal{S}_{i,r}$, excluding the first c default indices (in other words, $P(s_{i,r} \leq c) = 0$). The permutation is uniformly random and independent of $\mathcal{S}_{i,r}$. Since we want to calculate an upper bound on the probability distribution of $s_{i,r}$, then we might assume w.l.o.g. that the set $\mathcal{U}_r$ has exactly $3f + 1 - c$ elements, and that the set $\mathcal{S}_{i,r}$ has $1.5f + 1 - c$, in both cases excluding units created by the c default nodes.

Then, for $s = 1, \dots, 1.5f + 1$ we have

$$P(s_{i,r} = s) = \frac{1.5f + 1 - c}{3f + 2 - c - s} \prod_{j=1}^{s-1} \frac{1.5f + 1 - j}{3f + 2 - c - j},$$

and $P(s_{i,r} > 1.5f + 1) = 0$, hence for $s = 1, 2, \dots$ we have

$$P(s_{i,r} = s) \leq 2^{-s+c}.$$

By Lemma C.9, all units from $\mathcal{S}_{i,r}$ must be decided 1, then to calculate $\text{ChooseHead}(r, \mathcal{D}_i)$ we need to wait for decisions on at most $s_{i,r} + c$ units of round r (here we include the default indices with highest priority). Using Fact C.1 for random variables from Lemma C.8 we see that for $K > 5$

$$\begin{aligned} P(Z_r \geq K \mid S_r = s) &\leq \sum_{k=1}^{s+c} P(Y(U_k) \geq K) \\ &\leq (s + c)(K - 1) \left(\frac{3}{4}\right)^{(K-8)/2}, \end{aligned}$$

therefore

$$\begin{aligned} P(Z_r \geq K) &\leq \sum_{s=1}^N P(S_r = s) P(Z_r \geq K \mid S_r = s) \\ &= O\left(K \cdot \left(\frac{3}{4}\right)^{K/2}\right). \end{aligned}$$

On the other hand, if the round is publicly forked, then we observe that we need to wait for decision about at most N^2 units of round r (since every unit can be forked at most N times), and use Fact C.1. $\square$

THEOREM C.12 (ORDERUNITS LATENCY [FOR QuickAleph]). Let W_r be a random variable representing the number of rounds required to append a head of round r to the linear order, assuming that the default indices for rounds $0, \dots, r$ are known. Formally, W_r is defined as the smallest l such that for every local copy $\mathcal{D}_i$ of height $r + l$ we have

$$\text{ChooseHead}(r, \mathcal{D}_i) \in \text{OrderUnits}(\mathcal{D}_i).$$

Then

$$\mathbb{E}(W_r) = O(1)$$

if the $\frac{4}{\log \frac{4}{3}} \log N$ rounds prior to r are not publicly forked, and

$$\mathbb{E}(W_r) = O(\log N)$$

otherwise.

PROOF. Let $\mathcal{P}_i$ be a node. $\text{ChooseHead}(r, \mathcal{D}_i) \in \text{OrderUnits}(\mathcal{D}_i)$ is equivalent to the statement that for $j = 0, 1, \dots, r$ we have

$$\text{ChooseHead}(j, \mathcal{D}_i) \neq \perp.$$

Therefore, if $W_r \geq K$, then for at least one round j we have $Z_j \geq K + r - j$, where Z_j is a random variable from Lemma C.11.

We start with the forked variant. Since

$$(W_r \geq K) \subseteq \bigcup_{j=0}^r (Z_j \geq K + r - j),$$

then

$$\begin{aligned}
P(W_r \geq K) &\leq \sum_{j=0}^r P(Z_j \geq K + r - j) \\
&\leq O(1) \cdot N^2 \sum_{j=0}^r (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} \\
&= O(1) \cdot N^2 \sum_{j=K}^{K+r} j \left(\frac{3}{4}\right)^{j/2} \\
&\leq O(1) \cdot N^2 \sum_{j=K}^{+\infty} j \left(\frac{3}{4}\right)^{j/2} \\
&= O\left(N^2 \cdot K \cdot \left(\frac{3}{4}\right)^{K/2}\right).
\end{aligned}$$

Note that above we used the fact that the constant hidden under the big O notation is independent of the round number.

Observe now that for $K > \frac{4}{\log \frac{4}{3}} \log N$

$$P(W_r \geq K - 4 \log \frac{3}{4} \log N) = O\left(K \cdot \left(\frac{3}{4}\right)^{K/2}\right),$$

and since W_r has values in $\mathbb{N}$, then

$$\mathbb{E}(W_r) = \sum_{K=1}^{+\infty} P(W_r \geq K) = O(\log N).$$

To analyze the non-forked case we make a very similar argument, but we use the tighter bound for the last $\frac{4}{\log \frac{4}{3}} \log N$ rounds. Denote

$$A := \frac{4}{\log \frac{4}{3}} \log N \text{ for brevity.}$$

$$\begin{aligned}
P(W_r \geq K) &\leq \sum_{j=0}^r P(Z_j \geq K + r - j) \\
&\leq O(1) \cdot \left(N^2 \sum_{j=0}^{r-A} (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} + \sum_{j=r-A+1}^r (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} \right) \\
&\leq O(1) \cdot \left(\sum_{j=A}^r (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} + \sum_{j=r-A+1}^r (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} \right) \\
&\leq O(1) \cdot \sum_{j=0}^r (K + j) \left(\frac{3}{4}\right)^{(K+j)/2} = O(1) \cdot \sum_{j=K}^{K+r} j \left(\frac{3}{4}\right)^{j/2} \\
&\leq O(1) \cdot \sum_{j=K}^{+\infty} j \left(\frac{3}{4}\right)^{j/2} = O\left(K \cdot \left(\frac{3}{4}\right)^{K/2}\right),
\end{aligned}$$

and therefore

$$\mathbb{E}(W_r) = \sum_{K=1}^{+\infty} P(W_r \geq K) = O(1).$$

□

Bounding the number of forked rounds. For completeness we prove that there can be only a small, finite number of publicly forked rounds for which the latency could be $O(\log N)$, and for

the remaining rounds (the default case) the latency is $O(1)$ as in Lemma C.11.

Note that according to the Definition C.2, when a dishonest node creates a single fork and shows it to one (or a small number) of honest nodes, this does not yet make the corresponding round publicly forked. Indeed, to increase the protocol latency the adversary must show the fork to the majority of honest nodes, and it must happen during the next 4 rounds. This definition captures a certain trade-off inherent in all possible forking strategies, namely if the attack is to increase the protocol latency, it cannot stay hidden from the honest parties.

DEFINITION C.3. We say that node i is a discovered forker, if there are two variants of the same unit U, U' created by i , and there is a round r , such that every round- r unit is above U and U' .

Being a discovered forker has two straightforward consequences. First, no unit of round r or higher can connect directly to units created by the forker, hence the node i is effectively banned from creating units of round higher than $r - 2$. Moreover, the round- r head unit is also above the fork, and thus we can easily introduce some consensus-based mechanisms of punishing the forker.

LEMMA C.13. Let $r \geq 0$ be a round number. Among all $2f + 1$ round- r units created by honest nodes there is at least one unit U_0 such that at least $f + 1$ units of round $r + 1$ created by honest nodes are above U_0 , and every unit of round $r + 2$ (ever created) is above U_0 .

PROOF. Recall that units created by honest nodes cannot be forked. Every honest unit of round $r + 1$ has at least $2f + 1$ parents created by distinct nodes, hence it has at least $f + 1$ parents created by honest nodes. By the pigeonhole principle, there exists at least one round- r unit U_0 , created by an honest node, such that it is a parent of at least $f + 1$ units of round $r + 1$ created by honest nodes (denote these units by $\mathcal{U}$). Now, every unit U of round $r + 2$ has at least $2f + 1$ parents created by distinct nodes, hence must have at least one parent in $\mathcal{U}$, which implies $U \geq U_0$. □

LEMMA C.14. Whenever a round is publicly forked then during the next 7 rounds at least one forker is discovered. Consequently, if f' denotes the total number of discovered forkers then at most $7f' = O(N)$ rounds of the protocol can be publicly forked.

PROOF. Let U_0 be the round- $(r+5)$ unit whose existence is proved in Lemma C.13. Then U_0 is above at least one of $f + 1$ fork witnesses, and therefore every unit of round $r + 7$ is above a fork created by some node i (of course, this could happen earlier). Since no unit of round $r + 7$ or higher can connect directly to units created by i , then no unit of round higher than $r + 5$ created by node i can be present in the ch-DAG. Indeed, every such unit must have children to be added to the local copy held by nodes other than i , and those children units would be invalid. Therefore the node i can create forks at six rounds, from r to $r + 5$, and these rounds can possibly be publicly forked, hence the $6f'$ upper bound. Note that node i could have forked at some publicly forked rounds with number less than r , but then some other malicious node must have been discovered. □

C.3 QuickAleph– The Optimistic Case

Here, we conduct an analysis of the QuickAleph protocol behavior in a partially synchronous scenario. It shows that the protocol apart from being capable of operating with constant latency in an asynchronous environment (as proved in the previous subsection), but also matches the validation time achieved by state-of-the-art synchronous protocols.

Since the protocol itself does not operate on the network layer, but on ch-DAG, for sake of this subsection we need to translate partial synchrony to the language of ch-DAGs.

DEFINITION C.4. *A ch-DAG is synchronous at round r if every unit of round r produced by an honest node has units of round $r - 1$ produced by all honest nodes as its parents.*

Note that in case of synchronous network the above condition can be enforced by restricting each honest node to wait Δ before producing the next unit. Since the protocol is secure even with no synchrony assumptions, the delay Δ can be adaptively adjusted to cope with partial synchrony in a similar manner as, say, in Tendermint.

THEOREM C.15. *If $\mathcal{D}$ is a ch-DAG that is synchronous in rounds $r + 1$ and $r + 2$ and $\text{DefaultIndex}(r, \mathcal{D})$ is honest, then any unit of round $r + 3$ is enough to compute the head at round r .*

PROOF. Let i be the DefaultIndex at round r and U be the unit created by $\mathcal{P}_i$ at round r . Since the ch-DAG is synchronous at round $r + 1$, all honest nodes create a unit that is above U and thus vote 1 on U . Further, again by synchrony, each unit V of round $r + 2$ created by an honest node is above at least $2f + 1$ units that are above U , hence V "sees" $2f + 1$ votes on 1 for U and since the common vote at this round is 1 we can conclude that

$$\text{UnitDecide}(U, V, \mathcal{D}) = 1.$$

Finally, at round $r + 3$ the unit U is returned as the first proposal for the head by the $\text{GeneratePermutation}$ function²³ and thus

$$\text{ChooseHead}(r, \mathcal{D}) = U.$$

□

D TRANSACTION ARRIVAL MODEL AND A PROOF OF THEOREM 2.1

We are interested in the strongest possible form of protocol latency, i.e., not only measuring time between the moment a transaction is placed in a unit and the moment when it is commonly output as a part of total ordering but instead to start the "timer" already when the transaction is input to a node. For this, we introduce the following model.

Transaction arrival model. We assume that nodes continuously receive transactions as input and store them in their unbounded buffers. We make the following assumption regarding the buffer sizes:

²³It may seem that the decision could have been made at round $r + 2$ already. Such an "optimization" though breaks safety of the protocol in case of a malicious proposer. Indeed, if the proposer forks the proposed unit, some node can decide it as head in round $r + 2$, while another node could miss this variant and instead decide a different variant on 1 later during the protocol execution. This cannot happen at round 3 by Lemma C.4

DEFINITION D.1 (UNIFORM BUFFER DISTRIBUTION). *The ratio between buffer sizes of two honest nodes at the time of creation of their respective units of a given DAG-round is bounded by a constant $C_B \geq 1$.*

While it may look as a limiting assumption, it is not hard to ensure it in a real-life scenario by requiring each transaction to be sent to a randomly chosen set of nodes. An adversary could violate this assumption by inputting many transactions to the system in a skewed manner (i.e., sending them only to several chosen nodes), but such attack would be expensive to conduct (since usually a small fee is paid for each transaction) and hence each node with such artificially inflated buffer could be considered Byzantine.

Including transactions in units. Assuming such a model, each honest node randomly samples $\frac{1}{N}$ transactions from its buffer to be included in each new unit. Additionally, it removes transactions that are already included in units of its local copy of ch-DAG from its buffer.

DAG-rounds and async-rounds. There are two different notions of a "round" used throughout the paper – DAG-round as defined in Section 3.1 and async-round, as defined by [19] and in Section G. While the DAG-round is more commonly used in the paper, it should be thought as a structural property of ch-DAGs and hence not necessarily a measure of passing time. Hence, to be in line with the literature, all proofs regarding latency are conducted in terms of async-rounds. The following lemma provides a useful one-sided estimate of unit production speed in terms of async-rounds, roughly it says that DAG-rounds (asymptotically) progress at least as quickly as async-rounds.

LEMMA D.1. *Each honest node initializes ch-RBC for a newly produced unit at least once during each 5 consecutive async-rounds.*

PROOF. Let $\mathcal{P}_i$ be an honest node. We prove that if $\mathcal{P}_i$ have instantiated ch-RBC for unit of DAG-round r_d at async-round r_a then it necessarily instantiates it again (for a different unit) before async-round $r_a + 5$ ends. We show it by induction on r_d .

If $r_d = 1$, then necessarily $r_a = 1$ since every node is required to produce a unit in its first atomic step. Then, by Lemma F.1(Latency), $\mathcal{P}_i$ will have at least $2f + 1$ units of DAG-round 1 in its local copy of ch-DAG by the end of async-round 4 and hence will be required to instantiate ch-RBC at that time, what concludes the proof in this case.

Now assume $r_d > 1$. To produce a unit of DAG-round r_d , there must have been at least $2f + 1$ units of DAG-round $r_d - 1$ in $\mathcal{D}_i$. Then at least $2f + 1$ instances of ch-RBC for units of that DAG-round have had to locally terminate for $\mathcal{P}_i$ before r_a . Hence, by Lemma F.1(Fast Agreement), we get that every honest node needs to have at least $2f + 1$ units of DAG-round $r_d - 1$ in its local copy of ch-DAG before async-round $r_a + 2$ ends. Consequently, each honest node is bound to instantiate ch-RBC for its unit of DAG-round r_d or higher before async-round $r_a + 2$ ends. We consider the following two cases.

Case 1. Every honest node has instantiated ch-RBC for a unit of DAG-round r_d before async-round $r_a + 2$ ended. Then, every honest node (including $\mathcal{P}_i$) will locally output the result of this ch-RBC instances before DAG-round $r_d + 5$ ends, by Lemma F.1(Latency).

Hence, $\mathcal{P}_i$ is bound to instantiate ch-RBC for a unit of DAG-round at least $r_d + 1$ before async-round $r_a + 5$ ends.

Case 2. If Case 1 does not hold, then by the above, at least one honest node has instantiated ch-RBC for a unit of DAG-round $r'_d > r_d$ before async-round $r_a + 2$ ended. Then, again using Lemma F.1 (Fast Agreement), we conclude that every honest node (including $\mathcal{P}_i$) needs to hold at least $2f + 1$ units of DAG-round $r'_d - 1 \geq r_d$ before async-round $r_a + 4$ ends. Then $\mathcal{P}_i$ is bound to instantiate ch-RBC for a unit of DAG-round at least $r_d + 1$ before async-round $r_a + 4$ ends. $\square$

Next, we formulate a lemma connecting the concept of async-round and the transaction arrival model described on the beginning of this section.

LEMMA D.2. *If a transaction tx is input to at least k honest nodes during async-round r_a then it is placed in an honest unit in async-round $r_a + O(\frac{N}{k})$.*

PROOF. By Lemma D.1 each of the k honest nodes with tx in buffer is bound to produce at least one unit per 5 async-rounds. Since the honest nodes are sampling transactions to be input to units independently, the probability of tx being input to at least one unit during any quintuple of async-rounds after r_a is at least

$$1 - \left(1 - \frac{1}{N}\right)^k > 1 - \left(1 - \frac{k}{N} + \frac{k^2}{2N^2}\right) = \frac{k}{N} - \frac{k^2}{2N^2} > \frac{k}{2N},$$

by extended Bernoulli's inequality, since $\frac{k}{N} \leq 1$.

Hence the expected number of async-rounds before tx is included in a unit is $O(\frac{N}{k})$. $\square$

We now proceed to proving two technical lemmas that are required to show that, on average, every transaction input to the system is placed in a unit only a small (constant) number of times. We begin with a simple probability bound that is then used in the second lemma.

LEMMA D.3. *Let $n, m \in \mathbb{N}$ and suppose that $Q_1, Q_2, \dots, Q_n$ are finite sets with $|Q_i| \geq m$ for every $i = 1, 2, \dots, n$. For every $i = 1, 2, \dots, n$ define a random variable $S_i \subseteq Q_i$ that is obtained by including independently at random each element of Q_i with probability $\frac{1}{n}$. If $m \geq 48n$ then*

$$P\left(\min_{T \subseteq [n], |T|=n/3} \left| \bigcup_{i \in T} S_i \right| \leq \frac{m}{6}\right) \leq e^{-n}.$$

PROOF. For convenience denote $Q = \bigcup_{i \in [n]} Q_i$. Let us fix any subset $T \subseteq [n]$ of size $n/3$ and denote $S^T = \bigcup_{i \in T} S_i$. For every element $q \in Q$ define a random variable $Y_q \in \{0, 1\}$ that is equal to 1 if $q \in S^T$ and 0 otherwise. Under this notation, we have

$$|S^T| = \sum_{q \in Q} Y_q.$$

We will apply Chernoff Bound to upper bound the probability of

$$\sum_{q \in Q} Y_q \leq \frac{m}{6}.$$

For this, note first that $\{Y_q\}_{q \in Q}$ are independent. Moreover, we have $\mathbb{E}\left[\sum_{q \in Q} Y_q\right] \geq \frac{m}{3}$ from linearity of expectation. Hence from the Chernoff Bound we derive

$$P\left(\left|S^T\right| \leq \frac{m}{6}\right) = P\left(\sum_{q \in Q} Y_q \leq \frac{m}{6}\right) \leq e^{-\frac{m}{24}} \leq e^{-2n}.$$

Finally, by taking a union bound over all (at most 2^n many) sets T of size $n/3$ we obtain

$$P\left(\min_{T \subseteq [n], |T|=n/3} |S^T| \leq \frac{m}{6}\right) \leq 2^n \cdot e^{-2n} = e^{-n}.$$

$\square$

LEMMA D.4. *Let T_R be the number of, not necessarily different, transactions (i.e., counted with repetitions) input to units during the first $R = \text{poly}(N)$ rounds of the protocol, and T'_R be the number of pairwise different transactions in these units, then $T_R = O(T'_R + RN)$ except with probability $e^{-\Omega(N)}$.*

PROOF. For every round r denote by B_r the common (up to a constant) size of this round's buffer of any honest node. Since every unit contains at most $O(B_r/N)$ transactions in round r , we have that the total number of transactions input in units this round is $O(B_r)$ and thus $T_R = O\left(\sum_{r=0}^R B_r\right)$.

From Lemma E.2 at every round r , there exists a set $\mathcal{S}_r$ of units at round r such that every unit of round at least $r + 3$ is above every unit in $\mathcal{S}_r$. Let also $\mathcal{H}_r \subseteq \mathcal{S}_r$ be any subset of at least $f + 1$ honest units and $\mathcal{T}_r$ be the set of all transactions in $\mathcal{H}_r$. Since for any r_1, r_2 such that $r_2 \geq r_1 + 3$ we have $\mathcal{H}_{r_1} \supseteq \mathcal{H}_{r_2}$ and thus $\mathcal{T}_{r_1} \cap \mathcal{T}_{r_2} = \emptyset$. In particular

$$T'_R \geq \left| \bigcup_{r=0}^R \mathcal{T}_r \right| \geq \frac{1}{3} \sum_{r=0}^R |\mathcal{T}_r|.$$

From Lemma D.3 we obtain that unless $B_r < 48N$ it holds that $|\mathcal{T}_r| \geq \frac{B_r}{6}$ with probability $1 - e^{-N}$. Thus finally, we obtain that with high probability (no matter how large B_r is) $|\mathcal{T}_r| \geq \frac{B_r}{6} - 8N$, and consequently

$$T'_R \geq \frac{1}{3} \sum_{r=0}^R |\mathcal{T}_r| \geq \frac{1}{18} \sum_{r=0}^R B_r - \frac{8}{3} RN,$$

and therefore

$$T_R = O\left(\sum_{r=0}^R B_r\right) = O(T'_R + RN).$$

$\square$

PROOF OF THEOREM 2.1.

Total Order and Agreement. It is enough to show that all the honest nodes produce a common total order of units in ch-DAG. Indeed, the transactions within units can be ordered in an arbitrary yet deterministic way, and then a consistent total order of units trivially yields a consistent total order of transactions. The total order is deterministically computed based on the function ChooseHead applied to every round, which is consistent by Lemma C.5 (Agreement).

Censorship Resilience and Latency. By Lemma D.2 we obtain that each transaction input to k honest nodes is placed in a unit U created by some honest node $\mathcal{P}_i$ after $O(N/k)$ async-rounds.

By Lemma F.1(Latency) each honest node receives U via ch-RBC within 3 async-rounds from its creation and hence will append it as a parent of its next unit, which will be created within 5 async-rounds by Lemma D.1 and added to ch-DAGs of all honest nodes within next 3 async-rounds, again by Lemma F.1(Latency). Thus, after $O(\frac{N}{k})$, the unit U containing tx is below all maximal units created by honest nodes. Denote by r_d the maximum DAG – round over all honest nodes at that moment and by r_a the current async-round.

Since each unit has at least $2f + 1$ parents, it follows that every unit of round $r_d + 1$ (even created by an dishonest node) is above U . For this reason, the head of round $r_d + 1$ is also above U and in particular the unit U is ordered at latest by the time when the head of this round is chosen. Note that by Lemma C.5(latency), the head at round $r_d + 1$ is determined after only $O(1)$ DAG-rounds. Combining this fact with Lemma D.1 it follows that after $5 \cdot O(1)$ async-round each honest node can determine the head from its version of the poset. We obtain that the total latency is bounded by

$$O(N/k) + O(1) \cdot O(1) = O(N/k).$$

Communication Complexity. Let t_r be the number of transactions that have been included in honest units at level r . By Lemma F.1(Communication Complexity) we obtain that the total communication complexity utilized by instances of ch-RBC over any sequence of R DAG-rounds is

$$O\left(N \sum_{r=1}^R t_r + RN^2 \log N\right) = O(T_R + RN^2 \log N),$$

where T_R is the total number of transactions included in units at all rounds $r = 0, 1, \dots, R$. From Lemma D.4 we conclude that $T_R = O(T'_R + RN)$ where T'_R is the number of distinct transactions in these units. The final bound on communication complexity is

$$O(T'_R + RN^2 \log N).$$

□

E RANDOMNESS BEACON

The main goals of this section are to show that ABFT-Beacon implements a Randomness Beacon (thus providing a proof of Theorem 2.2), and to prove correctness of two implementations of SecretBits that are provided in Section 4, i.e., prove Lemma 4.1 and Lemma 4.2.

Let us now briefly summarize what is already clear and what is left to prove regarding the randomness beacon.

- (1) Every node $\mathcal{P}_i$ reliably broadcasts its key box KB_i (in $U[i; 0]$) that contains a commitment to a polynomial A_i (of degree at most f) and encrypted tossing keys for all the nodes. Some of the tossing keys might be incorrect if $\mathcal{P}_i$ is dishonest.
- (2) For every $i \in [N]$ the unit $U[i; 6]$ defines a set $T_i \subseteq [N]$ and the i th MultiCoin is defined as a threshold signature

scheme as if it were generated using the polynomial

$$B_i(x) = \sum_{j \in T_i} A_j(x).$$

- (3) A threshold signature with respect to MultiCoin_i , i.e., $m^{B_i(0)}$ for some $m \in G$ can be generated by simply multiplying together the threshold signatures $m^{A_j(0)}$ for $j \in T_i$.
- (4) Thus, on a high level, what remains to prove is that:
 - For every $i \in [N]$ such that $U[i; 6]$ is available, it is possible for the nodes to commonly generate threshold signatures with respect to key set KS_j for every $j \in T_i$.
 - For every $i \in [N]$ the key $B_i(0)$ remains secret, or more precisely, the adversary cannot forge signatures $m^{B_i(0)}$ for a randomly chosen $m \in G$.

Below, we proceed to formalizing and proving these properties.

Properties of Key Sets. From now on we will use the abstraction of Key Sets to talk about pairs (TK, VK) where

$$TK = (A(1), A(2), \dots, A(N))$$

$$VK = (g^{A(1)}, g^{A(2)}, \dots, g^{A(N)})$$

for some degree $\leq f$ polynomial $A \in \mathbb{Z}_q[x]$. In the protocol, the vector VK is always publicly known (computed from a commitment or several commitments) while TK is distributed among the nodes. It can be though distributed dishonestly, in which case it might be that several (possibly all) honest nodes are not familiar with their corresponding tossing key. In particular, every Key Box defines a key set and every MultiCoin defines a key set.

In the definition below we formalize what does it mean for a key set to be "usable" for generating signatures (and thus really generating randomness)

DEFINITION E.1 (USABLE KEY SET). We say that a Key Set $KS = (TK, VK)$ is usable with a set of key holders $T \subseteq [N]$ if:

- (1) every honest node is familiar with VK and T ,
- (2) $|T| \geq 2f + 1$,
- (3) for every honest node $\mathcal{P}_i$, if $i \in T$, then i holds tk_i .

The next lemma demonstrates that indeed a usable key set can be used to generate signatures.

LEMMA E.1. Suppose that (TK, VK) is a usable key set, then if the nodes being key holders include the shares for a nonce m in the ch-DAG at round r , then every honest node can recover the corresponding signature σ_m at round $r + 1$.

PROOF. Since the key set is usable, we know that there is exactly one underlying polynomial $A \in \mathbb{Z}_q[x]$ of degree f that was used to generate (TK, VK) , hence every unit at round r created by a share dealer $i \in T$ can be verified using VerifyShare and hence it follows that any set of $f + 1$ shares decodes the same value $\sigma_m = m^{A(0)}$ using ExtractShares (see also [6]). It remains to observe that among a set of (at least $2f + 1$) round- r parents of a round- $(r + 1)$ unit, at least $f + 1$ are created by nodes in T and thus contain appropriate shares. □

Generating Signatures using MultiCoins. We proceed to proving that the nodes can generate threshold signatures with respect

to MultiCoins. The following structural lemma says, intuitively, that at every round r there is a relatively large set of units that are below **every** unit of round $r + 3$.

LEMMA E.2 (SPREAD). *Assume that an honest node $\mathcal{P}_i$ has just created a unit U of round $r + 3$. At this moment, a set $\mathcal{S}_r$ of $2f + 1$ units of round r is determined such that for every unit V of round $r + 3$ and for any unit $W \in \mathcal{S}_r$, we have $W \leq V$.*

PROOF. Let K be a set of $2f + 1$ nodes that created units in $\downarrow(U)$. Let $\mathcal{T}$ be a set of $2f + 1$ units of round $r + 1$ created by nodes in K . Every unit in $\downarrow(U)$ is above $2f + 1$ units of round $r + 1$, hence it is above at least $f + 1$ units in $\mathcal{T}$ (the remaining f units may be created by nodes outside of K). By the pigeonhole principle, there exists a unit $U_0 \in \mathcal{T}$ such that at least $f + 1$ units in $\downarrow(U)$ are above it. Thus, every subset of $2f + 1$ units of round $r + 2$ must include at least one unit above U_0 , so every valid unit V of round $r + 3$ will have at least one parent above U' , thus $V \geq U'$. Finally, choose $\mathcal{S}_r := \downarrow(V)$. $\square$

We now proceed to proving a structural lemma that confirms several claims that were made in Subsection 4.3 without proofs. We follow the notation of Subsection 4.3.

LEMMA E.3. *Let T_i for $i \in [N]$ be the sets of indices of Key Sets forming the i th MultiCoin, then*

- (1) *For every i such that $U[i; 6]$ exists, T_i has at least $f + 1$ elements.*
- (2) *For every $i \in [N]$ and every $j \in T_i$, the Key Set KS_j is usable.*
- (3) *For every unit V of round $r + 1$ with $r \geq 6$, there exists a set of $f + 1$ units in $\downarrow(V)$ such that they contain all shares required to reconstruct all Key Sets that are below V .*

PROOF. Let $\mathcal{D}$ here be the local view of an honest node that has created a unit of round at least 6. From Lemma E.2 we obtain sets $\mathcal{S}_0$ and $\mathcal{S}_3$, which are in $\mathcal{D}$ because it includes at least one unit of round 6.

For any $i \in [N]$ such that $U[i; 6]$ exists in $\mathcal{D}$, the set T_i is defined as a function of the unit $U[i; 6]$ and all units below it, therefore it is consistent between honest nodes. Additionally, $U[i; 6]$ is above at least $f + 1$ units from $\mathcal{S}_0$ created by honest nodes ($2f + 1$ units minus at most f created by malicious nodes), and creators of those units must be included in T_i . Indeed, if a unit of round zero was created by an honest node, then it is valid, and every round-3 unit must vote 1 on it, as false-negative votes cannot be forged.

From now on, let T denote the union of all T_i s that are well defined (i.e., exist) in $\mathcal{D}$. Every unit W in the set $\mathcal{S}_3$ must vote 1 on every Key Box KB_l for $l \in T$, because W is seen by every unit of round 6. Since $|\mathcal{S}_3| \geq 2f + 1$ and due to the definition of votes, we see that every Key Set KS_l for $l \in T$ is usable by (possibly a superset of) creators of $\mathcal{S}_3$.

If we take the intersection of the set of creators of $\mathcal{S}_3$, and the set of $\downarrow(V)$ creators, then the resulting set contains at least $f + 1$ nodes. Units in $\downarrow(V)$ created by these $f + 1$ nodes must contain valid shares from round r to all Key Sets created by nodes from T . Indeed, this is required by the protocol, since these nodes declared at round 3 that they will produce valid shares (recall that $f + 1$ valid shares are enough to read the secret).

Finally we remark that the node who holds $\mathcal{D}$ might not be familiar with $\mathcal{S}_3$, but, nevertheless, it can find at least $f + 1$ units in $\downarrow(V)$ containing all required shares, because it knows T . $\square$

In part (3) of the above lemma we show that we can always collect shares from a single set of $f + 1$ units, which yields an optimization from the viewpoint of computational complexity. The reason is that each round of the setup requires generating $O(N)$ threshold signatures, but if we can guarantee that each signature has shares generated by the same set of $f + 1$ nodes, then we may compute the Lagrange coefficients for interpolation only once (as they depend only on node indices that are in the set) thus achieve computational complexity $O(N^3)$ instead of $O(N^4)$ as in the basic solution.

Security of MultiCoins. The remaining piece is unpredictability of multicoin, which is formalized below as

LEMMA E.4. *If a polynomially bounded adversary is able to forge a signature generated using MultiCoin $_{i_0}$ (for any $i_0 \in [N]$) for a fresh nonce m with probability $\varepsilon > 0$, then there exists a polynomial time algorithm for solving the computational Diffie-Hellman problem over the group G that succeeds with probability ε .*

PROOF. We start by formalizing our setting with regard to the encryption scheme that is used to encrypt the tossing keys in key boxes. Since it is semantically secure we can model encryption as a black box. More specifically we assume that for a key pair (sk, pk) , an adversary that does not know sk has to query a black box encryption oracle $\text{Enc}_{pk}(d)$ to create the encryption of a string d for this public key. Further, we assume that whenever a string s is fed into the decryption oracle $\text{Dec}_{sk}(\cdot)$ such that s was not obtained by calling the encryption oracle by the adversary, then the output $\text{Dec}_{sk}(\cdot)$ is a uniformly random string of length $|s|$. Since the key pairs we use in the protocol are dedicated, the adversary has no way of obtaining ciphertexts that were not created by him, for the key sets he is forced to use.

Given this formalization we are ready to proceed with the proof. We fix any $i_0 \in [N]$ and consider the MultiCoin $_{i_0}$. We use a simulation argument (see e.g. [43]) to show a reduction from forging signatures to solving the Diffie-Hellman problem. To this end, let (g^α, g^β) be a random input to the Diffie-Hellman problem, i.e., the goal is to compute $g^{\alpha\beta}$.

Suppose that an adversary can forge a signature of a random element $h \in G$ with probability $\varepsilon > 0$ when the honest nodes behave honestly. We now define a **simulated run** of the protocol (to distinguish from an **honest run** in which all honest node behave honestly) which is indistinguishable from an honest run from the perspective of the adversary, yet we can transform a successful forgery into a solution of the given Diffie-Hellman triplet.

Description of the simulated run. For convenience assume that the nodes controlled by the adversary have indices $1, 2, \dots, f$. At the very beginning, every honest node $i \in \{f + 1, f + 2, \dots, N\}$ generates uniformly random values $r_{i,0}, r_{i,1}, \dots, r_{i,f} \in \mathbb{Z}_q$ and defines a polynomial A_i of degree at most f by specifying its values

at $f + 1$ points as follows:

$$\begin{aligned} A_i(0) &= \alpha + r_{i,0}, \\ A_i(1) &= r_{i,1}, \\ &\vdots \\ A_i(f) &= r_{i,f}, \end{aligned}$$

where $\alpha \in \mathbb{Z}_q$ is the unknown value from the Diffie-Hellman instance. Note that since α is unknown the i th node has no way to recover the polynomial $A_i(x) = \sum_{j=0}^f a_{i,j}x^j$ but still, it can write each of the coefficients in the following form

$$a_{i,j} = a_{i,j,0} + \alpha a_{i,j,1} \quad \text{for } j = 0, 1, \dots, f,$$

where $a_{i,j,0}, a_{i,j,1} \in \mathbb{Z}_q$ are known values. Since g^α is known as well, this allows the i th node to correctly compute the commitment

$$C_i = (g^{a_{i,0}}, g^{a_{i,1}}, \dots, g^{a_{i,f}}),$$

pretending that he actually knows A_i . Similarly he can forge a key box KB_i so that from the viewpoint of the adversary it looks honest. This is the case because $\mathcal{P}_i$ knows the tossing keys for the adversary $tk_1, tk_2, \dots, tk_f$ since these are simply $r_{i,1}, r_{i,2}, \dots, r_{i,f}$. Thus $\mathcal{P}_i$ can include in E_i the correct values $e_{i,1}, e_{i,2}, \dots, e_{i,f}$ and simply fill the remaining slots with random strings of appropriate length (since these are meant for the remaining honest nodes which will not complain about wrong keys in the simulation).

In the simulation all the honest nodes are requested to vote positively on key boxes created by honest nodes, even though the encrypted tossing keys are clearly not correct. For the key boxes dealt by the adversary the honest nodes act honestly, i.e., they respect the protocol and validate as required. Suppose now that the setup is over and that the honest nodes are requested to provide shares for signing a particular nonce m . The value of the hash $h = \text{hash}(m) \in G$ is sampled in the simulation as follows: first sample a random element $\delta_m \in \mathbb{Z}_q$ and subsequently set

$$\text{hash}(m) = g^{\delta_m}.$$

Since in the simulation the value δ_m is revealed to the honest nodes, the honest node $\mathcal{P}_j$ can compute its share corresponding to the i th key set (for $i \geq f + 1$) as

$$h^{A_i(j)} = (g^{\delta_m})^{A_i(j)} = (g^{A_i(j)})^{\delta_m},$$

where the last expression can be computed since $g^{A_i(j)}$ is $vk_{i,j}$ (thus computable from the commitment) and δ_m is known to j .

Solving Diffie-Hellman from a forged signature. After polynomially many rounds of signing in which the hash function is simulated to work as above, we let the hash function output g^β (the element from the Diffie-Hellman input) for the final nonce m' for which the adversary attempts forgery. The goal of the adversary is now to forge the signature, i.e., compute $(g^\beta)^{A(0)}$, where

$$A(0) = \sum_{i \in T_{i_0}} A_i(0),$$

is the cumulative secret key for the multicoin MultiCoin_{i_0} . For that, the adversary does not see any shares provided by honest nodes. Note that if we denote by H and D the set of indices of honest and

dishonest nodes (respectively) within T_{i_0} then the value of $A(0)$ is of the following form

$$\begin{aligned} A(0) &= \sum_{i \in T_{i_0}} A_i(0) \\ &= \sum_{i \in H} A_i(0) + \sum_{i \in D} A_i(0) \\ &= |H| \cdot \alpha + \sum_{i \in H} r_{i,0} + \sum_{i \in D} A_i(0). \end{aligned}$$

We observe that each value $A_i(0)$ for $i \in D$ can be recovered by honest nodes, since at least $f + 1$ honest nodes must have validated their keys when voting on KB_i and thus honest nodes collectively know at least $f + 1$ evaluations of A_i and thus can interpolate $A_i(0)$ as well. Similarly, the values $r_{i,0}$ for $i \in H$ are known by honest nodes. Finally we observe that since by Lemma E.3 (a), $|T_i| \geq f + 1$, it follows that $|H| > 0$. Thus consequently, we obtain that $A(0)$ is of the form

$$A(0) = \gamma\alpha + \mu,$$

where $\gamma, \mu \in \mathbb{Z}_q$ are known to the honest nodes and $\gamma \neq 0$. Hence, the signature forged by the adversary has the form

$$(g^\beta)^{A(0)} = g^{\gamma\alpha\beta + \mu\beta} = (g^{\alpha\beta})^\gamma \cdot (g^\beta)^\mu,$$

and can be used to recover $g^{\alpha\beta}$ since γ, μ and g^β are known.

The above reasoning shows that we can solve the Diffie-Hellman problem given an adversary that can forge signatures; however to formally conclude that, we still need to show that the view of the adversary in such a simulated run is indistinguishable from an honest run.

Indistinguishability of honest and simulated runs. We closely investigate all the data that is emitted by honest nodes throughout the protocol execution to see that these distributions are indistinguishable between honest and simulated runs. Below we list all possible sources of data that adversary observes by listening to honest nodes.

- (1) The key boxes in an honest run and a simulated run look the same to the adversary, as they are always generated from uniformly random polynomials, and the evaluations at points $1, 2, \dots, f$ are always honestly encrypted for the nodes controlled by the adversary. The remaining encrypted keys are indistinguishable from random strings and thus the adversary cannot tell the difference between honest runs and simulated runs, since the values encrypted by honest nodes are never decrypted.
- (2) The votes (i.e., the $\text{VerKey}(\cdot, \cdot)$ values) of honest nodes on honest nodes are always positive, hence the adversary does not learn anything here.
- (3) The shares generated by honest nodes throughout the execution of the protocol are always correct, no matter whether the run is honest or simulated (in which case the honest nodes do not even know their keys).
- (4) The votes of honest nodes on dishonest key boxes: this is by far the trickiest to show and the whole reason why we need dedicated key pairs for encryption in key boxes. We argue that the adversary can with whole certainty predict

the votes of honest nodes, hence this piece of data is insignificant: intuitively the adversary cannot discover that he "lives in simulation" by inspecting this data. Here is the reason why the adversary learns nothing from these votes. There are two cases:

- (a) a dishonest node $i \in [f]$ includes as $e_{i,j}$ some ciphertext that was generated using the encryption black box $\text{Enc}_{i \rightarrow j}$. That means that the adversary holds a string d such that $e_{i,j} = \text{Enc}_{i \rightarrow j}(d)$. In this case the adversary can itself check whether d is the correct tossing key. Thus the adversary can predict the vote of $\mathcal{P}_j$: either it accepts if d is the correct key, or it rejects in which case it reveals $\text{Dec}_{i \rightarrow j}(e_{i,j}) = d$.
- (b) a dishonest node $i \in [f]$ includes as $e_{i,j}$ some string that was never generated by the encryption black box $\text{Enc}_{i \rightarrow j}$. In this case, the decryption of $e_{i,j}$ is simply a random string and thus agrees with the key only with negligible probability. Thus the adversary can tell with whole certainty that the vote on $e_{i,j}$ will be negative and he only learns a random string $\text{Dec}_{i \rightarrow j}(e_{i,j})$.

All the remaining data generated by honest nodes throughout the execution of the protocol is useless to the adversary as he could simulate it locally. This concludes the proof of indistinguishability between honest and simulated runs from the viewpoint of the adversary. $\square$

Proofs regarding SecretBits and ABFT-Beacon. We are now ready to prove Lemma 4.2, i.e. show that the implementation of SecretBits with MultiCoin during the ABFT – Beacon setup satisfies the Definition 3.3 of SecretBits.

PROOF OF LEMMA 4.2. Let $\mathcal{P}_k$ be any honest node. We start by showing that during the run of the ABFT – Beacon setup if $\mathcal{P}_k$ invokes $\text{SecretBits}(i, r)$, then $r \geq 10$ and the unit of round 6 created by $\mathcal{P}_i$ is already present in the local copy $\mathcal{D}_k$.

All SecretBits invocations happen inside ChooseHead(6) or, more precisely, in GeneratePermutation(6, $\mathcal{D}$) and CommonVote($U, \cdot$), for units U of round 6. Let $\mathcal{U}$ be a set of units of round 6 present in $\mathcal{D}_k$ at the time of any particular call to ChooseHead(6). Additionally, let C be the set of all nodes that created units in $\mathcal{U}$. The procedure GeneratePermutation(6, $\mathcal{D}$) calls $\text{SecretBits}(i, 6+4)$ for all $i \in C$, and function CommonVote(U, V) is deterministic if $R(V) \leq R(U) + 4$ and hence calls only $\text{SecretBits}(i, k)$, where i is U 's creator in C and $k \geq 10$.

Thus, from the above, the function SecretBits will be invoked sufficiently late, hence all units of round 6 defining required MultiCoins will be already visible at that time. Lemma E.3 guarantees that then the MultiCoin can be used to generate signatures since all the keys sets comprising it are usable. The secrecy follows from the fact that the signatures generated by MultiCoins are unpredictable for the adversary (Lemma E.4). $\square$

The above proof justifies why is it easier to implement SecretBits as a function of two arguments, instead of only using the round number.

The next lemma shows that by combining key sets as in our construction, we obtain a single key set that is secure, i.e., that

there are enough nodes holding tossing keys and that the generated bits are secret.

Given the above lemma, we are ready to conclude the proof of Theorem 2.2.

PROOF OF THEOREM 2.2. From Lemma 4.2 we obtain that the Setup phase of ABFT – Beacon executes correctly and a common Key Set (TK, VK) is chosen as its result. Moreover, by Lemma E.4 the output Key Set is secure and thus can be used to generate secrets. Consequently, Setup + Toss indeed implement a Randomness Beacon according to Definition 2.2.

Communication Complexity. What remains, is to calculate the communication complexity of the protocol. Observe that during the setup, every unit might contain at most:

- $O(N)$ hashes of parent nodes,
- $O(N)$ verification keys,
- $O(N)$ encrypted tossing keys,
- $O(N)$ votes,
- $O(N)$ secret shares.

Since ChooseHead builds only an expected constant number of DAG-rounds before terminating (see Lemma C.5), every node reliably broadcasts $O(1)$ units during the setup in expectation each of which of size $O(N)$, thus the total communication complexity per node is $O(N^2 \log N)$.

Latency. Assume that honest nodes decide to instantiate Toss for a given nonce m during async-round r_a . Then, each honest node multicasts its respective share before async-round $r_a + 1$ ends, thus all such shares are bound to be delivered before the end of async-round $r_a + 2$. Thus, each honest node may reveal the value of a Toss after at most 2 async-round, what concludes the proof. $\square$

The last remaining piece is to prove Lemma 4.1 that claims correctness of the main implementation of SecretBits.

PROOF OF LEMMA 4.1. The case with a trusted dealer follows for instance from [6]. Consider now the harder case: when the Setup of ABFT – Beacon is used to deal the keys. Then, the nodes end up holding a Key Set that is secure by Lemma E.4 and can be used for signing messages by Lemma E.1. $\square$

F RELIABLE BROADCAST

In this section we describe ch-RBC – a modified version of Reliable Broadcast (RBC) protocol introduced by [9] and improved by [17] by utilizing erasure codes. The protocol is designed to transfer a message from the sender (who instantiates the protocol) to the set of receiving nodes in such a way that receivers can be certain that they all received the same version of the message. More formally, a protocol implements Reliable Broadcast if it meets the following conditions:

- (1) **Termination.** If an honest node instantiates RBC, all honest nodes eventually terminate.
- (2) **Validity.** If an honest node instantiates RBC with a message m , then any honest node can only output m .
- (3) **Agreement.** If an honest node outputs a message m , eventually all honest nodes output m .

We implement four rather straightforward modifications to the original RBC protocol. First, before multicasting prevote message

each node checks the size of the proposed unit (line 10) to prevent the adversary from sending too big units and hence consuming too much bandwidth of honest nodes²⁴. Second, before multicasting prevote, we wait until ch-DAG of a node reaches at least one round lower than round of prevoted unit (line 11). This check counters an attack in which adversary initializes ch-RBC for meaningless pieces of data of rounds which do not exist in the ch-DAG yet, hence wasting bandwidth of honest nodes. Third, after reconstructing a unit U , we check if U is valid (line 16), i.e., we check if it has enough parents, if one of its parents was created by U 's creator, and if the data field of U contains all required components. Finally, before multicasting commit message for unit U , nodes wait until they receive all parents of U via RBC (line 17) to ensure that it will be possible to attach U to ch-DAG²⁵.

The following lemma summarizes the most important properties of ch-RBC protocol. Note that **Latency** is a strictly stronger condition than **Termination**, and similarly **Fast Agreement** is stronger than **Agreement**. Thus, in particular, this lemma proves that ch-RBC implements Reliable Broadcast.

LEMMA F.1. (*reliable broadcast*) For any given round r , all ch-RBC runs instantiated for units of that round have the following properties:

- (1) **Latency.** ch-RBC instantiated by an honest node terminates within three async-rounds for all honest nodes.
- (2) **Validity** If an honest node outputs a unit U in some instance of ch-RBC, U had to be proposed in that instance.
- (3) **Fast Agreement.** If an honest node outputs a unit U at async-round r_a , all honest nodes output U before async-round $r_a + 2$ ends.
- (4) **Message Complexity.** In total, all the ch-RBC instances have communication complexity²⁶ at most $O(t_r + N^2 \log N)$ per node, where t_r is the total number of transactions input in honest units of round r .

PROOF. **Latency.** Let $\mathcal{P}_i$ be an honest node instantiating ch-RBC for unit U_i during the async-round r so that the propose messages are sent during r as well. Each honest node during the atomic step in which it receives the propose message multicasts the corresponding prevote. Since proposals were sent during async-round r , async-round $r + 1$ cannot end until all of them are delivered, all honest nodes multicast prevotes during async-round $r + 1$. Similarly, all prevotes sent by honest nodes need to reach their honest recipients during async-round $r + 2$, hence honest nodes multicast their commits during async-round $r + 2$ ²⁷. Finally, the commits of all honest nodes reach their destination before the end of async-round $r + 3$ causing all honest nodes to locally terminate at that time, what concludes the proof.

²⁴Since in this version of RBC nodes learn about the content of broadcast message only after multicasting it as prevote, without this modification adversary could propose arbitrarily big chunks of meaningless data and hence inflate message complexity arbitrarily high.

²⁵Note that this condition is not possible to check before multicasting prevote since nodes are not able to recover U at that stage of the protocol.

²⁶We remark that $N^2 \log(N)$ can be improved to N^2 in the communication complexity bound by employing RSA accumulators instead of Merkle Trees in the RBC protocol.

²⁷A cautious reader probably wonders about the **wait** in line 16. It will not cause an execution of ch-RBC instantiated by an honest node to halt since before sending proposals during async-round r_a a node must have had all U 's parents output locally by other instances of ch-RBC, hence each other node will have them output locally by the end of async-round $r_a + 2$, by condition **Fast Agreement**

ch-RBC:

```

/* (Protocol for node  $\mathcal{P}_i$  with sender  $\mathcal{P}_s$ 
   broadcasting unit  $U$  of round  $r$ ) */
1 if  $\mathcal{P}_i = \mathcal{P}_s$  then
2    $\{s_j\}_{j \in N} \leftarrow$  shares of  $(f+1, N)$ -erasure coding of  $U$ 
3    $h \leftarrow$  Merkle tree root of  $\{s_j\}_{j \in N}$ 
4   for  $j \in N$  do
5      $b_i \leftarrow$  Merkle branch of  $s_j$ 
6     send propose( $h, b_j, s_j$ ) to  $\mathcal{P}_j$ 
7 upon receiving propose( $h, b_j, s_j$ ) from  $\mathcal{P}_s$  do
8   if received_propose( $\mathcal{P}_i, r$ ) then
9     terminate
10  if check_size( $s_j$ ) then
11    wait until  $\mathcal{D}_i$  reaches round  $r - 1$ 
12    multicast prevote( $h, b_j, s_j$ )
13    received_propose( $\mathcal{P}_i, r$ ) = True
14 upon receiving  $2f + 1$  valid prevote( $h, \cdot, \cdot$ ) do
15   reconstruct  $U$  from  $s_j$ 
16   if  $U$  is not valid then terminate
17   wait until all  $U$ 's parents are locally output by  $\mathcal{P}_i$ 
18   interpolate all  $\{s'_j\}_{j \in N}$  from  $f + 1$  shares
19    $h' \leftarrow$  Merkle tree root of  $\{s'_j\}_{j \in N}$ 
20   if  $h = h'$  and commit( $\mathcal{P}_s, r, \cdot$ ) has not been send then
21     multicast commit( $\mathcal{P}_s, r, h$ )
22 upon receiving  $f + 1$  commit( $\mathcal{P}_s, r, h$ ) do
23   if commit( $\mathcal{P}_s, r, \cdot$ ) has not been send then
24     multicast commit( $\mathcal{P}_s, r, h$ )
25 upon receiving  $2f + 1$  commit( $\mathcal{P}_s, r, h$ ) do
26   output  $U$  decoded from  $s_j$ 's
27 check_size( $s_j$ ):
28    $T \leftarrow$  number of transactions in  $U$  inferred from  $s_j$ -th size
29   /*  $C_B$  is a global parameter of the protocol */
30   if  $T > C_B \cdot (i\text{-th batch size in round } r)$  then
31     output False
32   else output True

```

Validity. The first honest node to produce the commit message for a given unit U has to do it after receiving $2f + 1$ precommit messages for U , out of which at least $f + 1$ had to be multicast by other honest nodes. Since honest nodes are sending prevotes only for shares they received as proposals, U had to be proposed by node initiating ch-RBC.

Fast Agreement. Assume that an honest node $\mathcal{P}_i$ has output a unit U at async-round r_a during some instance of ch-RBC. Since it can happen only after receiving $2f + 1$ commit messages for U , at least $f + 1$ honest nodes had to multicast commit for U . Each honest node engages in ch-RBC for only one version of each node's unit for a given DAG-round, so it is not possible for any other node to output a unit different than U for that instance of ch-RBC.

On the other hand, $\mathcal{P}_i$ must have received at least $f + 1$ commit messages for U , which have been multicast before async-round r_a ended. By definition of async-round, all of these commits must have reached their honest recipients before async-round $r_a + 1$ ended.

Consequently, after gathering $f + 1$ commits for U , all honest multicasted their commits before async-round $r_a + 1$ ended. Thus, each honest node received commits from every other honest node before the end of async-round $r_a + 2$, and terminated outputting U .

Message Complexity. Each honest node engages in at most N ch-RBC instances for each DAG-round r , since it accepts only one proposal from each node. Let now $\mathcal{P}_i$ be an honest node with buffer of size B_r . Due to the condition in line 10, $\mathcal{P}_i$ engages only in ch-RBC instances broadcasting units of size at most $C_B \frac{B_r}{N}$, where C_B is the constant from the *uniform buffer distribution* assumption.

A single instance of ch-RBC proposing a unit U has communication complexity of $O(\text{size}(U) + N \log N)$, see [17] (here the communication complexity is computed *per honest node*). Now, because of the *uniform buffer distribution* property we know that every valid unit of round r includes between $C_B^{-1} \frac{B_r}{N}$ and $C_B \frac{B_r}{N}$ transactions (with $C_B = \Theta(1)$) thus if U is such a unit, we have

$$\text{size}(U) = O\left(\frac{B_r}{N} + N\right),$$

where the $+N$ comes from the fact that such a unit contains $O(N)$ parent hashes. Consequently the total communication complexity in round r is upper bounded by

$$\begin{aligned} \sum_{i=1}^N (\text{size}(U[i; r]) + N \log N) &= N \cdot O\left(\frac{B_r}{N} + N + N \log N\right) \\ &= O(B_r + N^2 \log N) \\ &= O(t_r + N^2 \log N). \end{aligned}$$

Where the last equality follows because the total number of transactions in honest units is $t_r = 2N/3 \cdot \Theta(B_r/N) = \Theta(B_r)$. $\square$

General RBC with proofs of termination. Note that the above scheme differs from classical RBC only in four mentioned ch-DAG-related aspects, hence after deleting additional checks it can be used to broadcast arbitrary pieces of data. Additionally, one simple modification can be done, which provides a proof that RBC has terminated for each node which had received the output locally. Namely, together with commit message, each node can send share of a threshold signature of hash h with threshold $2f + 1$. Then, each node that gathered $2f + 1$ is able to construct the threshold signature and use it as a proof of the fact that RBC had finalized at its end.

G ASYNCHRONOUS MODEL

Slightly informally, the difference between synchronous and asynchronous network is typically explained as the absence of a global bound Δ on all message delays in asynchronous setting. In this paper, we use the following formalization adopted from Canetti and Rabin [19], where the adversary has a full control over the network delays, but is committed to eventually deliver each message to its recipient.

DEFINITION G.1 (ASYNCHRONOUS NETWORK MODEL). *The execution of the protocol proceeds in consecutive atomic steps. In each*

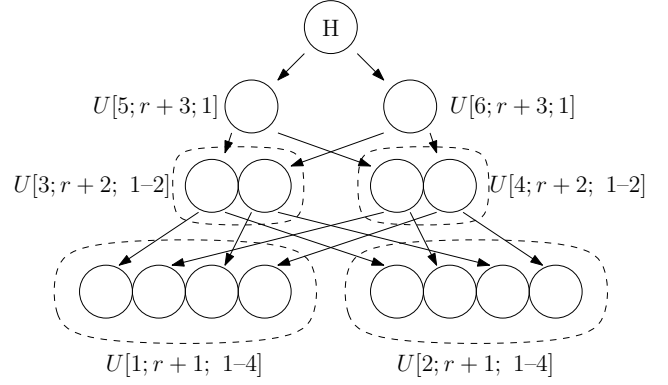


Figure 1: A "Fork Bomb" for $K = 3$. Only the top unit (marked H) is created by an honest node. Every collection of units within a dashed oval depicts forks produced by a certain dishonest node.

atomic step the adversary chooses a single node $\mathcal{P}_i$ which is then allowed to perform the following actions:

- *read a subset S , chosen by the adversary, among all its pending messages. Messages in S are considered delivered,*
- *perform a polynomially bounded computation,*
- *send messages to other nodes; they stay pending until delivered.*

Thus, in every atomic step one of the nodes may read messages, perform computations, and send messages. While such a model may look surprisingly single-threaded, note that an order in which nodes act is chosen by an adversary, hence intuitively corresponds to the worst possible arrangements of network delays. The following definition by [19] serves as a main measure of time for the purpose of estimating latency of our protocols.

DEFINITION G.2 (ASYNCHRONOUS ROUND). *Let l_0 be an atomic step in which the last node is chosen by adversary to perform an action, i.e., after l_0 each node acted at least once. The asynchronous rounds of the protocol and the subsequent l_i are defined as follows:*

- *All atomic steps performed after l_{i-1} until (and including) l_i have asynchronous round i ,*
- *l_i is the atomic step in which the last message sent during asynchronous round l_{i-1} is delivered.*

H FORK BOMB ATTACK

In this section we describe a resource-exhaustion attack scenario on DAG-based protocols. Generally speaking, this attack applies to protocols that allows adding nodes (which in our case are called units – we stick to this name for the purpose of explaining the attack) to the DAG performing only “local validation”. By this we mean that upon receiving a unit, the node checks only that its parents are correct and that the unit satisfies some protocol-specific rules that are checked based only on the data included in this unit and on its parents (possibly recursively), and adds this unit to the DAG if the checks are successful. In particular, such a protocol has to allow adding forked units to the DAG, as it can happen that two distinct honest nodes built upon two distinct variants of a fork,

hence there is no way to roll it back. At this point we also remark that a mechanism of banning nodes that are proved to be forking (or malicious in some other way) is not sufficient to prevent this attack.

We note that while the QuickAleph protocol (without the alert system) satisfies the above condition, the Aleph protocol and the QuickAleph extended with the alert system, do not. This is because Aleph uses reliable broadcast to disseminate units (which gives a non-local validation mechanism) and similarly the alert system for QuickAleph adds new non-local checks before adding units created by forkers. On the other hand, protocols such as Hashgraph [4] or Blockmania [23] satisfy this condition and thus are affected, if not equipped with an appropriate defense mechanism.

The main idea of the attack is for a group of malicious nodes to create an exponentially large number of forking units, which all must be accepted by honest nodes if the protocol relies only on locally validating units. The goal is to create so many valid forked units that honest nodes which end up downloading them (and they are forced to do so since these units are all valid) are likely to crash or at least slow down significantly because of exhausting RAM or disc memory. Even though this leads to revealing the attacking nodes, there might be no way to punish them, if most of the nodes are unable to proceed with the protocol.

Description of the attack. We describe the attack using the language and rules specific to the QuickAleph protocol (without the alert system). For simplicity, when describing parents of units we only look at their dishonest parents and assume that apart from that, they also connect to all honest units of an appropriate round. At the end of this section we also give some remarks on how one can adjust this description to any other DAG-based protocol.

Technically, the attack is performed as follows: for K consecutive rounds all malicious nodes create forking units, but abstain from sending them to any honest node, after these K rounds, they start broadcasting forks from this huge set. Assume that the malicious nodes participating in the attack are numbered $1, 2, \dots, 2K - 1, 2K$. Recall that $U[j; r]$ denotes the round- r unit created by node j . We extend this notation to include forks – let $U[j; r; i]$ denote the i -th fork, hence $U[j; r] = U[j; r; 1]$.

Suppose it is round r when the attack starts, it will then finish in round $r + K$. We refer to Figure 1 for an example of this construction for $K = 3$. A formal description follows: fix a $k \in \{1, 2, \dots, K\}$, the goal of nodes $2k - 1$ and $2k$ is to fork at round $r + k$, more specifically:

- (1) For the initial $k - 1$ rounds (starting from round r) these nodes create regular, non-forking units.
- (2) At round $r + k$ the nodes create 2^{K-k} forks each, which are sent only to other malicious nodes if $k < K$, and broadcast to every other node if $k = K$. Note that nodes $2K - 1$ and $2K$ in fact do not fork, hence their units must be eventually accepted by honest nodes.
- (3) Units $U[1; r + 1; i]$ and $U[2; r + 1; i]$ can have any round- r units as parents, for $i = 1, \dots, 2^{G-1}$.
- (4) For $k > 1$ unit $U[2k - 1; r + k; i]$ has $U[2k - 3; r + k - 1; 2i - 1]$ and $U[2k - 2; r + k - 1; 2i - 1]$ as parents, for $i = 1, \dots, 2^{K-k}$.
- (5) For $k > 1$ unit $U[2k; r + k; i]$ has $U[2k - 3; r + k - 1; 2i]$ and $U[2k - 2; r + k - 1; 2i]$ as parents, for $i = 1, \dots, 2^{K-k}$.

Validity of all the forks is easily checked by induction: all units $U[1; r + 1; i]$ and $U[2; r + 1; i]$ are valid, because there are no forks below them (although they are invalid in the sense that they are forked, this fact always becomes apparent only after some other valid nodes has built upon them). Now assume that units $U[2k - 3; r + k - 1; 2i - 1]$ and $U[2k - 2; r + k - 1; 2i - 1]$ are valid. Then the unit $U[2k - 1; r + k; i]$ is valid, because there are no forks created by nodes $2k - 3$ or $2k - 2$ below it. Analogously, the unit $U[2k; r + k; i]$ is valid.

Since the nodes $2K - 1$ and $2K$ cannot be banned, then units $U[2K - 1; r + K; 1]$ and $U[2K; r + K; 1]$ with their lower cones must be accepted by all honest nodes, and both of them contain at least $2^{K+1} - 2$ units in total, counting only K most recent rounds.

To conclude let us give a couple of remarks regarding this attack:

- (1) The attack is rather practical, as it does not require any strong assumptions about the adversary (e.g. no control over message delays is required). It is enough for the adversary to coordinate its pool of malicious nodes.
- (2) The attack allows any number of parents for a unit. If the protocol requires exactly two parents per unit, with the previous unit created by the same node (as in Hashgraph) then one round of the attack can be realized in two consecutive rounds.
- (3) If a larger number of parents is required, then the adversary can connect in addition to any subset of honest units. This is possible because honest nodes can always advance the protocol without the help of malicious nodes, and the number of required parents cannot be higher than the number of honest nodes.